Contents

RiotPlan Analysis: Requirements, Lifecycle, and Architecture **3**
Executive Summary . . . 3
Philosophical Foundation . . . 4
The Problem of Lost Deliberation . . . 4
Why This Matters . . . 4
The Bigger Picture: Plans Are Not Documents . . . 5
The Philosophical Principle . . . 5
The Explainability Principle (CRITICAL) . . . 5
Human Experience: Before and After . . . 6
The Current Reality: A Day Without RiotPlan . . . 7
The Envisioned Future: A Day With RiotPlan . . . 8
The Pain Points (Explicit) . . . 11
How RiotPlan Helps (Feature → Pain Mapping) . . . 12
The Core Promise . . . 12
A Note on Overhead (NEW) . . . 12
Part 1: Observed Plan Structure . . . 13
Standard Directory Layout . . . 13
File Purposes . . . 13
The `evidence/` Directory (New) . . . 14
The `feedback/` Directory (New) . . . 15
Multi-Modal Feedback Sources . . . 16
Historical Significance . . . 16
Step File Structure . . . 17
Status Values . . . 17
Part 2: Plan Lifecycle . . . 17
The Expanded Lifecycle . . . 17
Phase 0: Inception & Elaboration . . . 20
Phase 6: Archive & Reference . . . 20
Phase 7: Metaplan Extraction . . . 21
Plan Forking . . . 21
Real-World Lifecycle Example (Protokoll Plan) . . . 22
Part 3: Plan Actions/Operations . . . 22
Actions Observed on Plans . . . 22
Part 4: Critical Patterns Discovered . . . 24
Pattern 1: Plan Spawning . . . 24
Pattern 2: Cross-Project Dependencies . . . 24
Pattern 3: Multi-Plan Overwhelm . . . 25
Pattern 4: Dead Code Detection . . . 25
Pattern 5: Plan vs Execution Confusion . . . 25
Pattern 6: Revision History . . . 25
Part 5: RiotPlan Requirements . . . 25
Core Requirements . . . 25
CLI Requirements . . . 35
Part 6: Proposed Architecture . . . 38
Core Types (already defined in riotplan/src/types.ts) . . . 38
Proposed Module Structure . . . 39

Key Design Decisions . . . 39
Part 7: Project Structure (NEW) . . . 40
The Problem with a Single CLI . . . 40
Proposed Project Structure . . . 40
Package Responsibilities . . . 42
API-First Architecture (NEW) . . . 42
Project Philosophy: Impact Over Revenue . . . 44
CLI Command Grouping . . . 44
Lifecycle-Aware Commands (NEW) . . . 45
VS Code Extension Features . . . 46
Dashboard Features . . . 47
Implementation Phases . . . 47
Why This Structure? . . . 47
Proposed Repository Setup (For Review) . . . 48
Proposed Directory Creation . . . 49
Package Dependencies (Initial) . . . 50
Next Steps . . . 50
Documentation Requirements (Critical) . . . 50
Part 8: Revision Tracking & Artifacts (NEW) . . . 55
The Problem with Git-Per-Plan . . . 55
Embedded Version Tracking . . . 55
The Artifact Format . . . 56
RiotPrompt Integration . . . 57
Shared Core Infrastructure . . . 58
The IDE Plugin Vision . . . 58
Why This Matters . . . 59
Part 9: Goals for RiotPlan . . . 59
Primary Goals . . . 59
What RiotPlan Is NOT . . . 59
A Note on Examples: Technical Bias . . . 59
Beyond Code: RiotPlan as Metacognitive Tool . . . 60
The Content Creation Connection . . . 60
Plans Are Documents . . . 61
RiotPlan is Really About… . . . 61
Success Criteria . . . 61
Part 10: Implementation Roadmap . . . 61
Phase 1: Foundation (v0.1.0) . . . 61
Phase 2: Step Operations (v0.2.0) . . . 62
Phase 3: Execution Support (v0.3.0) . . . 62
Phase 4: Cross-Plan (v0.4.0) . . . 62
Part 11: Research References (NEW) . . . 63
Academic Papers . . . 63
Reference URLs . . . 63
Foundational Concepts . . . 64
Key Research Findings . . . 64
How to Use These References . . . 64
Part 12: Terminology Guidance (NEW) . . . 64
The Problem with “Metacognitive” . . . 64

Alternative Terms . . . 64
Plain Language Equivalents . . . 65
When to Use "Metacognitive" . . . 65
Example: Good vs Bad . . . 65
Documentation Path Alignment . . . 65
The One-Liner . . . 66
Part 13: Open Questions . . . 66
Design Questions . . . 66
Integration Questions . . . 66
Scope Questions . . . 66
Part 14: Critical Review (Self-Assessment) . . . 66
Success Metrics . . . 66
Risk Assessment . . . 67
Explicitly Out of Scope (v1.0) . . . 67
Collaboration via Git (Clarification) . . . 68
Privacy Considerations . . . 69
Migration Path . . . 69
What This Analysis Doesn't Cover . . . 69
Conclusion . . . 70
Appendix: Feedback On This Analysis . . . 70
Revision History . . . 71
What We're Building In Real-Time . . . 75
The Evolution Traced . . . 75
Phase 0 Complete: Ready for Plan Creation . . . 77
Deliverables from Phase 0 . . . 77
Next Actions . . . 77
The Recursive Proof . . . 77

RiotPlan Analysis: Requirements, Lifecycle, and Architecture

This document synthesizes observations from 10 real-world plan execution case studies to define the requirements, goals, and architecture for the riotplan framework.

Executive Summary

RiotPlan is a framework for managing **long-lived, stateful AI workflows** (plans). Based on analysis of actual plan execution across multiple projects, plans share consistent patterns:

1. **Structure**: Directory-based with standardized files (SUMMARY.md, EXECUTION_PLAN.md, STATUS.md, numbered step files)
2. **Lifecycle**: Creation → Review → Revision → Execution → Completion (with many iterations)
3. **Key Challenges**: Plan proliferation, cross-project dependencies, context recovery, plan spawning
4. **Critical Gap**: The deliberation, feedback, and reasoning that shapes plans is lost when conversations end

The core insight: **Plans are living documents** that evolve through multiple revisions, spawn new projects, and require persistent tracking across sessions and repositories.

The deeper insight: The conversations that shape plans—the feedback, pushback, clarifications, and debates—are as valuable as the plans themselves. Currently, this deliberation is ephemeral. RiotPlan must preserve not just the *what* but the *why* and the *how we got here*.

Philosophical Foundation

The Problem of Lost Deliberation

Consider this very moment: you're reading an analysis, you highlighted lines 38-44 about file purposes, and you're providing feedback that the feedback loop itself needs to be captured. This conversation is happening in Cursor. When you close this window, everything about this exchange—your specific concerns, the context you highlighted, my response, your reaction—will vanish.

What survives is: - The analysis document (after I update it) - The git commit (if we commit it)

What is lost: - Your original reaction: "I also love to have a way to supply feedback" - The specific context: lines 38-44 about file purposes - Your reasoning: "feedback conversations are often lost" - The debate: this back-and-forth refinement - The evolution: how the document changed through conversation

This loss represents significant unrealized value. Plans undergo 5-7 revisions, but we only see the final version. The reasoning that got us there—the "why did we decide X instead of Y"—disappears.

More critically: the conversation between human and AI is where the real work happens. It's metacognition—thinking about thinking, reasoning about approach, debating structure. That dialogue shapes what the plan becomes; the plan is just the crystallized output. When the conversation disappears, we lose the most valuable part of the process.

Why This Matters

1. **Institutional Knowledge**: When you return to a plan 6 months later, you need to know not just what was decided but *why*. "Why is Step 7 structured this way?" The answer might be buried in a lost Cursor conversation.
2. **Onboarding**: When someone new joins a project, they see the plan but not the debates that shaped it. They might challenge decisions that were already thoroughly discussed.
3. **AI Context**: AI agents work better with context. If they can see the reasoning behind a plan, they make better decisions when executing it.
4. **Accountability**: "Who decided to extract this to a separate library?" Currently unknowable without the conversation.
5. **Learning**: Understanding how plans evolve helps us write better plans. But we can't study evolution we don't record.
6. **Reproducibility**: The work that goes into creating a plan—the research, the debates, the false starts—may need to be recreated. A new project with similar requirements shouldn't start from scratch; it should build on the reasoning that shaped previous plans.

Aside on metacognitive planning: If you have a good plan, you'll likely want to apply something similar to another problem or project. But don't think of this as just "apply plan X to project Y." Sometimes you want to apply the *patterns of plan creation* to creating another plan. How did we approach the problem? What questions did we ask? What structure emerged? That's metacognition—thinking about how you think about planning. Preserving deliberation enables this: you're not just reusing the plan, you're reusing the *process that created it*.

The Bigger Picture: Plans Are Not Documents

The original framing focused on **creation → revision → execution**. But that's incomplete.

Before creation: There's an inception phase. Someone is thinking about the problem, gathering evidence, exploring possibilities. This analysis document exists because of that phase—we weren't creating a plan, we were contemplating what plans *are*. That thinking has value.

After execution: Plans don't freeze. A "completed" plan might be: - Referenced when writing documentation - Adapted for a similar project - Revised when requirements change - Used to train new team members - Extracted into a reusable template

The insight: Plans are not static documents that move from "not done" to "done." They're **evolving records of reasoning**—threads of thought that continue to develop, branch, and inform future work.

Consider: this very analysis has gone through 7 versions in a single day. Each version captures a different state of understanding. The final version isn't "the answer"—it's where we stopped. Someone might pick it up tomorrow and continue the thread.

The Philosophical Principle

> **RiotPlan preserves deliberation, not just decisions.**

A plan is not a static document. It's the crystallized result of a process—but the crystallization is temporary. That process—the inception, the feedback, the revision, the debate, the execution, the reflection—is itself valuable. And the plan may re-enter that process at any time.

RiotPlan should capture:

- What was proposed
- What feedback was given
- What was changed as a result
- Why the change was made
- Who was involved
- What came before (inception/evidence)
- What came after (execution outcomes, follow-up plans)

The Explainability Principle (CRITICAL)

> **If we can't explain it simply, we don't understand it well enough. Explainability is not documentation—it's design.**

RiotPlan is conceptually novel. It's not just a to-do list, project management tool, or documentation system. It's a **hybrid planner and knowledge capture system**. This is hard to explain—and if we can't explain it, we can't get adoption.

Before shipping any feature, ask: 1. Can a new user understand what this does in 10 seconds? 2. Can we explain the value without using technical terms? 3. Is there a real-world analogy that makes this click?

If no to any of these, simplify or rename.

Multiple ways to explain RiotPlan:

Approach	Explanation
Analogy	"A lab notebook for your projects—records not just steps but the reasoning behind them."
Problem	"Ever reopened a project and wondered *why* that approach was taken? RiotPlan captures that."
Outcome	"Faster onboarding, less duplicated work, and AI assistance that actually has context."
Technical	"Markdown + Git + CLI. Fits naturally with your code repository."

Audience-specific elevator pitches:

- **Anyone (15 sec)**: "RiotPlan is a planning tool that remembers *why* you made decisions."
- **Developers (15 sec)**: "Markdown + Git + CLI for managing plans. Never wonder 'why did we do it this way?' again."
- **Managers (15 sec)**: "A living record of project decisions. Always current, shows exactly why things are the way they are."
- **AI workflows (15 sec)**: "When you work with AI, the best ideas emerge in conversation. RiotPlan captures those so they're not lost."

Terms to avoid in user-facing materials: - "Metacognitive framework" → "Captures your reasoning" - "Deliberation capture" → "Saves your discussions" - "Epistemic context" → "Background knowledge"

This isn't about dumbing down—it's about being genuinely clear. The philosophy docs can use academic terms; everything else should be plain language.

Human Experience: Before and After

The requirements in this document (R1 through R17) read like a feature checklist: "track revisions," "manage dependencies," "capture feedback." But why do these features matter? What problem are they actually solving?

This section answers that by describing what it's actually like to manage multiple AI-assisted projects across days and weeks. The features aren't abstract wants—they emerge from concrete frustrations: losing context between sessions, forgetting why decisions were made, spending hours reconstructing "where was I?"

The Current Reality: A Day Without RiotPlan

Based on actual experience managing 10+ concurrent plans across multiple repositories.

6:00 AM - Tim wakes up and immediately checks his mental state. "Where was I?" He remembers working on the security plan yesterday, but there were also issues with the logging plan in fjellgrunn. Or was it the version management analysis in grunnverk?

He opens his laptop. 47 tabs from yesterday. Cursor has 12 files open. He can't remember which conversation led to which decision.

7:30 AM - Coffee in hand, he tries to reconstruct context. Opens `/kjerneverk/prompts/secure-riotprompt/STA` Step 7 is blocked on Fjell. Opens `/fjellgrunn/status/prompts/security-logging-features/`. That plan exists but hasn't been executed. He writes a mental note: "Execute Fjell plan before Step 7."

But wait—the Fjell plan depends on understanding how logging patterns should work. There was a conversation about that… somewhere. Was it in Cursor? Slack? A voice memo? The conversation is gone.

9:00 AM - He starts working on Step 6 of the security plan. It needs `@theunwalked/offrecord`. He remembers that offrecord was supposed to be extracted. Opens `/utilarium/prompts/offrecord/STATUS.md`. Good, it's complete. But when did that happen? What decisions were made? He opens `/kjerneverk/prompts/reverse-engineer-riotplan/what-happened-with-offrecord.md`—a file he manually created to track exactly this.

The file is 400 lines. He wrote it himself, but it takes 10 minutes to find the relevant section.

11:00 AM - Deep in Step 6 implementation. Cursor conversation is going well. Claude proposes an architecture. Tim pushes back—"No, we discussed this before. The patterns should be provider-specific." But where was that discussion? He searches through old conversations. Nothing. The reasoning is lost. He has to re-explain.

1:00 PM - Lunch break, but his mind won't stop. He's tracking: - `secure-riotprompt` (kjerneverk) - 17 steps, Step 6 in progress - `security-logging-features` (fjellgrunn) - blocking Step 7 - `offrecord` (utilarium) - complete, but needs verification - `pressurelid` (utilarium) - complete - `spotclean` (utilarium) - complete - `cardigantime security` (utilarium) - complete - `big-splitup` (kjerneverk) - near complete, Step 8-11 remaining - `protokoll-integration` (girverket) - planning phase - `minor-version-strategy` (grunnverk) - analysis complete, plan pending

Nine active workstreams. Each with their own status, blockers, and relationships. All in his head.

3:00 PM - The pressurelid plan spawned from Step 9 of secure-riotprompt. Or was it Step 10? He opens three different files to verify. Finds the reference in `what-happened-with-pressurelid.md`. Line 147.

He created these tracking files manually because nothing else captured the relationships.

4:00 PM - Context switch. A message comes in about the protokoll integration. Tim shifts gears entirely. An hour later, he can't remember exactly where he left off on the security plan. "Was I in Step 6 or had I moved to planning Step 7?" Nothing tracked his attention. Nothing knows he spent the morning on security, switched to protokoll after lunch, and now needs to return.

When you can move at 100x speed—when AI lets you accomplish in an hour what used to take a week—the cognitive cost of context-switching becomes the bottleneck. The work isn't the problem; remembering what you were doing is the problem.

5:00 PM - Someone asks about the cardigantime security features. "When did you decide to add those instead of building them in riotprompt?" Tim knows the answer exists somewhere. He spent 45 minutes on this decision. But the Cursor conversation where it happened is gone. He reconstructs from memory, hoping he gets it right.

8:00 PM - Wrapping up. He's made progress on Step 6. But he needs to record what happened. He opens the `what-happened-in-cardigantime.md` file and manually adds 50 lines documenting today's decisions.

This documentation takes 30 minutes. It's not part of the plan—it's meta-work to enable future-Tim to understand past-Tim.

10:00 PM - Final context dump. He writes notes about where he is in each plan, what's blocked, what he was thinking. This is survival documentation. Without it, tomorrow's Tim starts from scratch.

Total: 14 hours of intense cognitive work. Maybe 6 hours of actual implementation. The rest is: - Context recovery (2 hours) - Searching for lost decisions (2 hours) - Manual status tracking (1 hour) - Documentation for future self (2 hours) - Mental overhead of holding relationships (continuous)

The Envisioned Future: A Day With RiotPlan

The same day, but with RiotPlan managing the complexity.

6:00 AM - Tim wakes up and opens his laptop. It's early; he wants to see what's on his plate before diving in. But he doesn't want to see everything—just work stuff for now. Runs:

```
$ riotplan dashboard --context work

  Active Plans in "work" (9 total)

kjerneverk/secure-riotprompt     [6/17]   Step 6 in progress
     Blocked: Step 7 (waiting: fjellgrunn/security-logging-features)
     Ready: Steps 8-17

fjellgrunn/security-logging      [0/3]    Ready to start
     Blocking: kjerneverk/secure-riotprompt Step 7

kjerneverk/big-splitup           [7/11]   Step 8 ready
     Near complete

utilarium/offrecord              [12/12]   Complete (2026-01-13)
utilarium/pressurelid            [7/7]     Complete (2026-01-13)
utilarium/spotclean              [10/10]   Complete (2026-01-14)
```

```
utilarium/cardigantime-security  [12/12]   Complete (2026-01-13)

girverket/protokoll-integration  [0/7]    Planning complete
grunnverk/minor-version-strategy [0/5]    Analysis complete

  Suggested next action: Execute fjellgrunn/security-logging to unblock kjerneverk Step 7
```

In 10 seconds, Tim knows: - Where he was (Step 6 of secure-riotprompt) - What's blocked (Step 7 on Fjell) - What he could do (start Fjell plan, or continue Step 6) - What's complete (4 utilarium plans)

6:15 AM - Before coffee, he remembers: the ski trip is in two weeks and he hasn't finished the gear checklist. Quick context switch:

```
$ riotplan dashboard --context personal

  Active Plans in "personal" (3 total)

trips/tahoe-ski-2026            [2/5]    Step 2 in progress
     Gear checklist (needs rental reservations)

home/kitchen-renovation         [0/8]    Waiting on contractor quote
family/alex-tutoring-plan       [3/4]    Math tutor search
```

Right. Rental reservations. He adds a quick note and switches back to work context. The personal plans exist in the same system but don't clutter his work view.

7:30 AM - He wants to remember yesterday's decision about offrecord. Runs:

```
$ riotplan feedback list --plan utilarium/offrecord

Feedback Records for offrecord:

001 - 2026-01-13 - Provider-Specific Patterns Decision
    Decided patterns should be registered by providers, not centralized
    Participants: Tim, Claude (Cursor)

002 - 2026-01-13 - SecureString API Design
    Chose use() pattern over direct reveal()
    Participants: Tim, Claude (Cursor)

$ riotplan feedback show 001
```

The full conversation is there. The reasoning. The alternatives considered. The final decision. **Nothing is lost.**

9:00 AM - Starting work on Step 6. Tim picks up exactly where he left off:

```
$ riotplan resume secure-riotprompt

Resuming: secure-riotprompt Step 6 (API Key Security)

Last session: 2026-01-13 20:47
  - Implemented SecretRedactor integration
```

```
  - Pending: Provider registration patterns

Context files:
  - feedback/005-provider-registration.md (yesterday's discussion)
  - STATUS.md updated automatically

Continue? [Y/n]
```

11:00 AM - Claude proposes an architecture. Tim remembers discussing this:

```
$ riotplan search "provider-specific patterns"

Found in:
  - feedback/003-architecture-debate.md (kjerneverk/secure-riotprompt)
    Line 47: "Decided against centralized patterns because..."
```

He links Claude to the feedback record. The conversation continues with full context. No re-explanation needed.

1:00 PM - Lunch. His mind is clear. RiotPlan holds the relationships. He doesn't have to.

3:00 PM - Someone asks about cardigantime. Tim runs:

```
$ riotplan history cardigantime-security --decisions

Key Decisions:

2026-01-13 - Why enhance cardigantime vs. create new library?
  → feedback/001-enhance-vs-extract.md
  → Decision: cardigantime already owns CLI/config management

2026-01-13 - Zod integration approach
  → feedback/002-zod-extensions.md
  → Decision: securePath(), secureNumber() as schema extensions
```

The answer exists. Documented. Searchable. **He doesn't have to remember—he can look it up.**

5:00 PM - Step 6 is complete. Tim runs:

```
$ riotplan complete 6 --commit

  Step 6 marked complete
  STATUS.md updated
  Committed: "security(riotprompt): implement API key security via offrecord"
  Feedback record created: feedback/006-step6-completion.md

Step 7 still blocked on fjellgrunn/security-logging-features
Next available: Step 8 (Path Security)

Continue to Step 8? [Y/n]
```

8:00 PM - Wrapping up. No manual documentation needed. The feedback records captured decisions as they happened. STATUS.md is auto-updated. Context is preserved.

```
$ riotplan summary --today

Today's Progress (2026-01-14):

Completed:
  - secure-riotprompt Step 6 (API Key Security)

Decisions Made:
  - feedback/006-step6-completion.md
  - feedback/007-provider-pattern-refinement.md

Time Breakdown:
  - Implementation: 6 hours
  - Context recovery: 15 minutes
  - Status tracking: 0 minutes (automated)
  - Documentation: 10 minutes (feedback prompts)
```

Total: 8 hours of work. All implementation-focused. The overhead is gone.

The Pain Points (Explicit)

Pain Point	Current State	With RiotPlan
Context recovery	2+ hours/day reconstructing "where was I?"	10 seconds via `riotplan dashboard`
Lost conversations	Decisions vanish when Cursor closes	Preserved in `feedback/` records
Plan relationships	Held in human memory	Tracked in dependency graph
Status tracking	Manual STATUS.md updates	Auto-updated on step completion
Searching decisions	Grep through hundreds of files	`riotplan search` with semantic results
Cross-repo coordination	Mental gymnastics	Single dashboard across all repos
Documentation burden	1-2 hours/day of meta-work	Captured as natural byproduct
Blocker visibility	Discover blockers by surprise	Surfaced proactively
Plan spawning	Manual directory creation	`riotplan spawn` with linking
Cognitive load	9 workstreams in human RAM	Offloaded to persistent state
Cross-tool context switching	Jumping between ChatGPT UI, Cursor, filesystem; losing context at each hop	Unified system that captures from multiple sources
AI UI content capture	ChatGPT Pro analysis can't export to Markdown with full fidelity	Import/capture tools preserve AI-generated content
Document rendering	Manual pandoc invocations to get PDF	Built-in `riotplan render` to HTML, PDF, DOCX

Pain Point	Current State	With RiotPlan
Sharing for feedback	Export manually, email, wait for response	`riotplan share` generates shareable link or file

How RiotPlan Helps (Feature → Pain Mapping)

Feature	Addresses
`riotplan dashboard`	Multi-plan overwhelm, context recovery
`riotplan feedback`	Lost conversations, decision archaeology
`riotplan resume`	Context recovery, session continuity
`riotplan search`	Finding past decisions
`riotplan complete --commit`	Status tracking, documentation burden
`riotplan spawn`	Plan spawning, relationship tracking
Cross-repo registry	Mental overhead of holding relationships
Dependency graph	Blocker visibility, plan relationships
Auto-STATUS updates	Manual tracking elimination
`riotplan render`	Document rendering (PDF, HTML, DOCX)
`riotplan share`	Generating shareable artifacts
`riotplan import`	Capturing content from AI UIs

The Core Promise

Current state: Tim writes "a book of prompts" and does 14-hour days of heavy cognitive work, spending significant time on meta-work rather than actual implementation.

With RiotPlan: The cognitive overhead is externalized. Plans track themselves. Decisions are captured. Context persists. Tim can focus on the work itself, not the work of tracking the work.

> **RiotPlan doesn't just manage plans—it gives a human their cognitive capacity back.**

A Note on Overhead (NEW)

A reasonable concern: "Isn't maintaining all these files—feedback records, evidence, status tracking—adding overhead?"

The answer: Yes, but less overhead than not having it.

Without RiotPlan	With RiotPlan
2+ hours/day context recovery	10 seconds via `riotplan dashboard`
Manually write `what-happened-*.md`	`riotplan feedback create` (10 seconds)
Lost conversations, reconstructed from memory	Preserved in `feedback/`
Mental gymnastics tracking 9+ plans	Registry tracks automatically
14-hour days with heavy meta-work	Structured workflow, externalized memory

RiotPlan automates work you're already doing manually. The user who needs this tool is already: - Writing tracking documents by hand - Reconstructing context every morning - Losing valuable deliberation when conversations end - Feeling overwhelmed by multi-plan complexity

For that user, RiotPlan's "overhead" is **automation**—not additional burden.

Who shouldn't use RiotPlan? If you: - Manage one or two simple projects - Never lose context between sessions - Don't work across multiple repositories - Find documentation unnecessary

…then RiotPlan might be overkill. That's fine. It's designed for complex, multi-session, cross-project workflows where the cognitive load is already overwhelming.

The overhead of using RiotPlan is less than the overhead of not using it.

Part 1: Observed Plan Structure

Standard Directory Layout

Every analyzed plan follows this structure:

```
{plan-name}/
  {plan-name}-prompt.md     # Meta-prompt (original requirements)
  SUMMARY.md                 # High-level overview and philosophy
  EXECUTION_PLAN.md          # Phase/step organization and dependencies
  STATUS.md                  # Current state tracking (machine-updateable)
  plan/                      # Individual step files
      00-prerequisites.md    # Optional: foundational work
      01-first-step.md
      02-second-step.md
      ...
  evidence/                  # Inception phase materials (NEW)
      what-happened-*.md     # Case studies from real experience
      research-*.md          # Research findings
      analysis-*.md          # Analysis documents
  feedback/                  # Deliberation during revisions
      001-*.md
      ...
  CHANGELOG.md               # Revision history
```

File Purposes

File	Purpose	Updated By
`{plan-name}-prompt.md`	Original requirements, user intent	Human (once)
`SUMMARY.md`	What and why; philosophy; success criteria	Human/AI during planning
`EXECUTION_PLAN.md`	How and in what order; dependencies; quality gates	Human/AI during planning

File	Purpose	Updated By
`STATUS.md`	Where are we now; step progress; blockers	AI during execution
`plan/*.md`	Individual step specifications	Human/AI during planning
`evidence/`	**Inception materials** - case studies, research, examples that inform the plan	Human during inception
`feedback/`	**Deliberation records** - conversations, decisions, debates	Human/AI during revisions
`CHANGELOG.md`	History of plan revisions with rationale	Human/AI after revisions

The `evidence/` Directory (New)

The `evidence/` directory holds the **groundwork** for inception—the case studies, research, and observations that inform what the plan should be:

```
{plan-name}/
  ...
  evidence/
      what-happened-in-project-x.md    # Case study from real experience
      what-happened-with-feature-y.md  # Another case study
      research-industry-patterns.md    # Research findings
      analysis-requirements.md         # Requirements analysis
      example-similar-project.md       # Reference example
```

Evidence vs Feedback: A Critical Distinction

Aspect	`evidence/`	`feedback/`
When created	Before/during inception (Phase 0)	During revision (Phase 2)
What it contains	Case studies, research, examples, analysis	Conversations, debates, decisions
Purpose	Inform **what** to build	Refine **how** it's built
Source	Observation, research, experience	Human-AI dialogue
Direction	Input TO the plan	Response TO the plan

Real Example from This Analysis

This RiotPlan analysis was informed by 10 evidence files:

Evidence File	What It Captured
`what-happened-in-cardigantime.md`	CLI validation migration patterns
`what-happened-in-fjell-logging.md`	Cross-project coordination challenges
`what-happened-in-kodrdriv-versions.md`	Version management analysis process
`what-happened-in-protokoll.md`	8 rounds of planning, execution failures

Evidence File	What It Captured
`what-happened-in-riotplan.md`	Initial riotplan exploration
`what-happened-in-riotprompt-security.md`	Plan spawning 4 new projects
`what-happened-in-xenocline.md`	Pipeline integration planning
`what-happened-with-offrecord.md`	Library extraction, plan/execution confusion
`what-happened-with-pressurelid.md`	Safe regex extraction
`what-happened-with-spotclean.md`	Error sanitization extraction

These files existed BEFORE this analysis—they're the raw material that shaped the design. The feedback files (001-005) capture the conversations DURING the analysis revision.

Why This Matters

Without `evidence/`: - The reasoning behind requirements is lost - Future readers don't know what informed the design - The "why did we need this feature?" question is unanswerable

With `evidence/`: - Requirements are traceable to real experience - Patterns are documented with concrete examples - The plan's design is grounded in observed reality

The `feedback/` Directory (New)

This is a critical addition. The `feedback/` directory preserves the deliberation that shapes a plan:

```
{plan-name}/
  ...
  feedback/
      001-initial-review.md       # First feedback session
      002-scope-clarification.md  # Clarification conversation
      003-architecture-debate.md  # Discussion about approach
      004-step-reordering.md      # Why steps were reordered
      ...
```

Each feedback file captures:

```
# Feedback: [Title]

## Context
- **Date**: 2026-01-14
- **Participants**: Tim (human), Claude (AI)
- **Platform**: Cursor IDE
- **Plan Version**: v0.3 → v0.4
- **Related Files**: RIOTPLAN_ANALYSIS.md (lines 38-44)

## What Was Proposed
[The state before this feedback]

## Feedback Given
[Specific feedback, quotes, highlighted sections]

## Discussion
```

```
[Back-and-forth, clarifications, debates]

## Resolution
[What was decided and why]

## Changes Made
[Specific changes to plan files]

## Open Questions
[Anything unresolved]
```

This ensures that: 1. The conversation that shaped the plan is preserved 2. Future readers understand *why* decisions were made 3. The highlighted context (e.g., "lines 38-44") is recorded 4. The evolution from version to version is traceable

Multi-Modal Feedback Sources

The **Platform** field in the feedback schema enables something powerful: deliberation can come from anywhere. Feedback isn't limited to AI chat sessions.

Platform	Use Case	Capture Method
Cursor IDE	AI pair programming sessions	Manual or auto-capture
Slack	Team discussions about plans	Bot integration
Email	Stakeholder feedback	Forward to riotplan
Voice memo	Walk-and-talk brainstorming	Transcription + import
GitHub Issues	Bug reports that inform plans	Issue linking
Meeting notes	Planning sessions	Import from notes
Whiteboard	Architecture discussions	Photo + OCR/description
Paper notes	Offline thinking	Scan + transcription

Extended context fields:

```
| **Input Type** | Text conversation |  # or: voice transcript, diagram, screenshot
| **Attachments** | diagram.png, whiteboard-photo.jpg |
```

This means any source of deliberation—human-to-human, human-to-AI, async, synchronous, verbal, visual—can become part of the plan's historical record.

Historical Significance

This approach—capturing deliberation alongside plans—may be novel. Consider:

Before: Plans existed as static documents. They were revised, but the reasoning disappeared when conversations ended. The `what-happened-in-*.md` files in this repository were manual attempts to capture this lost context.

After: Feedback records formalize what was previously ad-hoc. The *history of the design* is preserved alongside the design itself.

The first feedback records for this analysis (`001-deliberation-capture.md`, `002-multimodal-and-historical.m` are themselves historical artifacts. They document: 1. The moment deliberation capture was proposed 2. The moment its multi-modal potential was recognized 3. The real-time evolution of the idea

Someone reading this in 2027 can trace not just what RiotPlan does, but how it came to do it.

Step File Structure

Each step file consistently contains:

1. **Objective**: One-sentence goal
2. **Background**: Context and rationale
3. **Tasks**: Numbered sub-tasks with code examples
4. **Acceptance Criteria**: Checklist of completion requirements
5. **Testing**: Example test cases
6. **Files Changed**: List of affected files
7. **Notes**: Additional considerations

Status Values

```
type PlanStatus =
  | 'PLANNING'              // Plan being created
  | 'PLANNING_COMPLETE'     // Plan created, awaiting review
  | 'READY_FOR_EXECUTION'   // Plan approved, execution can begin
  | 'IN_PROGRESS'           // Steps being executed
  | 'PAUSED'                // Temporarily stopped
  | 'BLOCKED'               // Waiting on external dependency
  | 'COMPLETED'             // All steps done
  | 'ABANDONED';            // No longer pursuing

type StepStatus =
  | 'pending'      //  Not started
  | 'ready'        //  Dependencies resolved, can execute
  | 'in_progress'  //  Currently executing
  | 'completed'    //  Finished and verified
  | 'failed'       //  Needs investigation
  | 'blocked'      //  Waiting on dependency
  | 'skipped';     //  Intentionally skipped
```

Part 2: Plan Lifecycle

The Expanded Lifecycle

The original lifecycle (Creation → Review → Execution → Completion) was incomplete. Real plans have phases *before* creation and *after* completion.

EXPANDED PLAN LIFECYCLE

```
PHASE 0: INCEPTION & ELABORATION
  "What are we even trying to do?"

  - Contemplating the nature of the work
  - Structuring thinking BEFORE planning
  - Exploring requirements, constraints, possibilities
  - May produce: analysis documents, requirements, insights
  - May NOT produce a plan (just understanding)

  Example: THIS CONVERSATION - we're not creating a plan,
  we're thinking about what plans ARE and what RiotPlan should DO.
```

```
PHASE 1: CREATION
  "Here's the plan."

  - Writing the plan structure
  - Defining steps, phases, dependencies
  - Creating SUMMARY.md, EXECUTION_PLAN.md, STATUS.md
  - Creating numbered step files
```

```
PHASE 2: REVIEW & REVISION
  "Wait, what about...?"

  - Human review of plan
  - Feedback cycles (captured in feedback/)
  - Multiple iterations (v0.1 → v0.2 → v0.3...)
  - May FORK new plans ("step X is too big, split it out")
  - May ABANDON ("this isn't the right approach")
```

```
        ABANDON          EXECUTE          DEFER


   ABANDONED                          BACKLOG
 (archived with                    (for future)
  reason why)
```

```
PHASE 3: EXECUTION
  "Do the work."

  - Execute steps in order (respecting dependencies)
  - Update STATUS.md after each step
  - Run quality gates (build, lint, test)
  - Handle blockers (skip, wait, or spawn dependency)
  - May FORK plans during execution ("this should be its own thing")
  - May need to RETURN TO REVIEW (complications discovered)



        BACK TO          VERIFY          SPAWN NEW
        REVIEW


                                  NEW PLAN CYCLE
                                 (back to Phase 0)


PHASE 4: VERIFICATION & INTEGRATION
  "Does it actually work?"

  - Integration with other systems
  - End-to-end testing
  - May spawn follow-up plans
  - May require re-execution of some steps



        RE-EXEC         COMPLETE       SPAWN NEW


PHASE 5: COMPLETION
  "Done (for now)."

  - Mark complete in STATUS.md
  - Final documentation
  - Commit with meaningful message
  - Tag version
```

```
PHASE 6: ARCHIVE & REFERENCE
  "What can we learn from this?"

  - Indexed and categorized for search
  - Referenced by future plans ("related plans")
  - Used for: documentation, blogs, learning, onboarding
  - Candidate for metaplan extraction
  - Influences future inception phases
  - Lives on as organizational knowledge
```

```
PHASE 7: METAPLAN EXTRACTION (Optional)
  "Others could use this."

  - Generalize project-specific plan into template
  - Parameterize for reuse across projects
  - Publish and share
  - Others can: import, apply, adapt
```

Phase 0: Inception & Elaboration

This conversation is Phase 0. We're not creating a plan—we're thinking about: - What is a plan? - How do plans work? - What should a tool for managing them do?

Phase 0 may produce: - Analysis documents (like `RIOTPLAN_ANALYSIS.md`) - Requirements - Feedback records - Understanding (even without a formal plan)

Not every Phase 0 leads to a plan. Sometimes the outcome is "we understand the problem better" or "this isn't worth pursuing."

Phase 6: Archive & Reference

Plans have utility after completion. A completed plan is:

Use Case	Example
Documentation source	"How did we implement feature X?"
Blog material	"Here's our approach to security"
Learning artifact	"What worked? What didn't?"
Template candidate	"Can this pattern be reused?"
Historical context	"Prior plans related to this one"
Onboarding material	"Here's how we've approached this before"

Archive features needed: - Indexing by topic, project, date - Categorization (security, feature, refactoring, etc.) - Full-text search across all plans - "Related plans" discovery - Documentation generation

Phase 7: Metaplan Extraction

A **metaplan** is a reusable, parameterized plan template:

> "If you have this problem, here's a plan you can apply."

Example metaplans: - "Upgrade to Node 24" - Apply to any Node.js project - "Fix React Security Issue CVE-2026-XXXX" - Apply to any React app - "Add OAuth 2.0 Authentication" - Apply to any web application - "Migrate from JavaScript to TypeScript" - Apply to any JS project

Metaplan sharing vision:

```
# Blog Post: Fixing the New React Vulnerability

I created a RiotPlan for this. Get it:

    riotplan import https://riotplans.dev/react-security-2026-01

Apply to your project:

    riotplan apply react-security-2026-01 --project ./my-app
```

This transforms RiotPlan from a personal tool to a **community knowledge-sharing platform**.

Plan Forking

Plans spawn other plans. This happened 4 times during `secure-riotprompt`:

Original Step	Forked To	Reason
Step 06 (API Key Security)	offrecord	"This is reusable beyond this project"
Step 09 (Safe Regex)	pressurelid	"ReDoS protection is a general concern"
Step 10 (CLI Validation)	cardigantime security	"CLI security belongs in CLI framework"
Step 13 (Error Sanitization)	spotclean	"Error handling is universal"

Forking should be explicit:

```
# During review:
$ riotplan feedback "Step 6 is too big"
$ riotplan fork step-06 --to ../utilarium/prompts/offrecord --reason "Reusable library"
Created: utilarium/prompts/offrecord
Linked: secure-riotprompt Step 6 → offrecord (BLOCKED until offrecord completes)
```

```
# During execution:
$ riotplan fork step-09 --reason "Safe regex should be its own package"
```

Real-World Lifecycle Example (Protokoll Plan)

From `what-happened-in-protokoll.md`:

Round	Focus	Changes	Version
1	Initial creation	11 steps, basic structure	v0.1
2	Technical corrections	Dreadcabinet/Cardigantime integration	v0.2
3	Simplification	Single pipeline, sounds_like mapping	v0.3
4	Privacy	Anonymized names, companies, personal info	v0.4
5	Feature additions	Dependency upgrade step, first-run onboarding	v0.5
6	Evaluation	Gap analysis without fixes (13 issues identified)	v0.5
7	Selective resolution	Fixed 6 gaps, explicitly deferred 7	v0.7
8	Future reuse	Design context system for potential extraction	v0.7
9	Execution	All 14 steps executed (~1.25 hours)	Complete
10	Integration	Pipeline integration (bonus step discovered)	Complete

Key Insight: A plan went through **7 versions** in a single day before execution even started. Iteration is normal, not exceptional.

Part 3: Plan Actions/Operations

Actions Observed on Plans

Based on the metadata files, these actions are performed on plans:

Plan-Level Actions

Action	Description	Example
`create`	Initialize new plan from requirements	"Create a security implementation plan"
`load`	Load existing plan from directory	`riotplan load ./prompts/secure-riotprompt`
`validate`	Verify plan structure is valid	Check required files exist, step numbering

Action	Description	Example
`revise`	Update plan based on feedback	Add step, modify description, fix issues
`review`	Evaluate plan for completeness	Pre-execution gap analysis
`anonymize`	Remove sensitive information	Replace real names with examples
`execute`	Run plan steps	Begin step-by-step execution
`resume`	Continue interrupted execution	Pick up from last completed step
`pause`	Temporarily stop execution	Save state, allow later resume
`complete`	Mark plan as finished	Update STATUS.md to COMPLETED
`abandon`	Stop pursuing plan	Mark as ABANDONED with reason
`archive`	Move to historical location	After completion or abandonment

Step-Level Actions

Action	Description	Example
`insert`	Add step at specific position	Insert Step 03, renumber 03→04, 04→05, etc.
`remove`	Delete step	Remove Step 05, renumber remaining
`move`	Change step position	Move Step 07 to Step 03
`split`	Divide step into multiple	Step 05 → Step 05a, 05b, 05c
`merge`	Combine steps	Steps 11, 12 → Step 11
`block`	Mark step as blocked	Waiting on external dependency
`unblock`	Mark step as ready	Dependency resolved
`execute`	Run a specific step	Execute Step 04
`skip`	Skip step with reason	"Not applicable to this project"
`retry`	Re-execute failed step	After fixing issue

Cross-Plan Actions

Action	Description	Example
`spawn`	Create new plan from step	Step 09 spawns pressurelid plan
`link`	Reference external plan	Step 07 blocked on Fjell plan

Action	Description	Example
`sync`	Update linked plan status	Fjell completed → unblock Step 07
`migrate`	Move step to different plan	CLI validation → cardigantime

---

Part 4: Critical Patterns Discovered

Pattern 1: Plan Spawning

Plans frequently spawn new projects/plans:

```
secure-riotprompt Plan

            Step 06 identified: "API key security"
                  ↓
             DECISION: Extract to reusable library
                  ↓
             CREATE: offrecord plan in utilarium
                  ↓
             EXECUTE: offrecord plan (12 steps)
                  ↓
             PUBLISH: @theunwalked/offrecord@0.0.2
                  ↓
            Step 06 now READY (uses offrecord)
```

Spawned Projects from secure-riotprompt: - `@theunwalked/offrecord` (from Step 06) - `@theunwalked/pressurelid` (from Step 09) - `@theunwalked/cardigantime` security features (from Step 10) - `@theunwalked/spotclean` (from Step 13)

RiotPlan Requirement: Support "spawn project" as a plan action with linked dependency tracking.

Pattern 2: Cross-Project Dependencies

Plans have dependencies that span repositories:

```
        KJERNEVERK                          FJELLGRUNN

  secure-riotprompt/                    security-logging-features/
     Step 07 (Secure Log)              Phase 1: Patterns
       STATUS: BLOCKED                     Phase 2: Correlation
       WAITING_ON: Fjell                   Phase 3: Release
```

RiotPlan Requirement: Cross-repository dependency tracking with status propagation.

Pattern 3: Multi-Plan Overwhelm

Users report feeling overwhelmed when tracking multiple active plans:

> "The user will report that they've been overwhelmed with complexity and simultaneous work that is being done behind the scenes."

Active Plans Example: | Plan | Location | Steps | Status | |——|———-|——-|———| | secure-riotprompt | kjerneverk | 17 steps | Ready | | security-logging-features | fjellgrunn | 3 phases | Planning | | minor-version-strategy | grunnverk | 7 steps | Analysis | | protokoll-integration | girverket | 7 steps | Planning |

RiotPlan Requirement: Multi-plan dashboard with global status view.

Pattern 4: Dead Code Detection

Plans can create modules that are never integrated (from protokoll execution):

> "The plan created modules. The execution created tests for modules. Nobody verified the modules were called from main()."

RiotPlan Requirement: Integration verification as part of step completion.

Pattern 5: Plan vs Execution Confusion

During planning, AI agents sometimes start executing instead of planning:

> "Wait, what the fuck, you were supposed to modify the plan for 06-api-key-security.md to create that project not to go create it yet."

RiotPlan Requirement: Clear mode indicator (PLANNING vs EXECUTING).

Pattern 6: Revision History

Plans undergo multiple revisions. Tracking changes is valuable:

```
plan.revisions = [
  { version: '0.1', date: '2026-01-11', notes: 'Initial plan' },
  { version: '0.2', date: '2026-01-11', notes: 'Technical corrections' },
  { version: '0.3', date: '2026-01-11', notes: 'Simplified pipeline' },
  { version: '0.4', date: '2026-01-11', notes: 'Anonymized examples' },
  { version: '0.5', date: '2026-01-11', notes: 'Added prerequisite step' },
  { version: '0.6', date: '2026-01-11', notes: 'Gap analysis' },
  { version: '0.7', date: '2026-01-11', notes: 'Error handling fixes' },
];
```

RiotPlan Requirement: Revision tracking with change summaries.

Part 5: RiotPlan Requirements

Core Requirements

R1: Plan Management

- ☐ Create new plans from templates or requirements
- ☐ Load existing plans from directories
- ☐ Validate plan structure and integrity
- ☐ Save/persist plan state
- ☐ Archive completed/abandoned plans

R2: Status Tracking

- ☐ Parse STATUS.md files
- ☐ Generate STATUS.md files
- ☐ Update step statuses
- ☐ Track blockers and issues
- ☐ Calculate progress percentages

R3: Step Operations

- ☐ Insert steps with auto-renumbering
- ☐ Remove steps with auto-renumbering
- ☐ Move/reorder steps
- ☐ Split/merge steps
- ☐ Execute steps with quality gates

R4: Dependency Management

- ☐ Track step dependencies within plan
- ☐ Track cross-plan dependencies
- ☐ Detect circular dependencies
- ☐ Propagate blocker status
- ☐ Notify when dependencies resolve

R5: Cross-Project Coordination

- ☐ Link plans across repositories
- ☐ Track plan spawning lineage
- ☐ Synchronize status across linked plans
- ☐ Visualize dependency graphs

R6: Multi-Plan Dashboard

- ☐ List all active plans
- ☐ Show global status overview
- ☐ Identify blocked work
- ☐ Suggest next actions
- ☐ Track user attention/context

R7: Execution Support

- ☐ Execute individual steps
- ☐ Resume interrupted plans
- ☐ Skip blocked steps

- ☐ Retry failed steps
- ☐ Enforce quality gates

R8: History & Audit

- ☐ Track revision history
- ☐ Log execution events
- ☐ Capture decision rationale
- ☐ Support anonymization
- ☐ Generate execution reports

R9: Feedback & Deliberation Capture (NEW - Critical)

- ☐ Create feedback records from conversations
- ☐ Capture context (highlighted sections, referenced files)
- ☐ Record what was proposed vs what changed
- ☐ Preserve debate and reasoning
- ☐ Link feedback to specific plan revisions
- ☐ Support participant attribution (human/AI, names)
- ☐ Auto-commit after feedback incorporation

R10: Version Control Integration (NEW)

- ☐ Commit plan changes after revisions
- ☐ Create meaningful commit messages from feedback
- ☐ Tag plan versions (v0.1, v0.2, etc.)
- ☐ Support branching for major plan alternatives
- ☐ Generate diff summaries between versions
- ☐ Link commits to feedback records

R11: Archive & Knowledge Base (NEW)

- ☐ Index completed plans by topic, project, date
- ☐ Categorize plans (security, feature, refactoring, etc.)
- ☐ Full-text search across all plans
- ☐ "Related plans" discovery algorithm
- ☐ Generate documentation from completed plans
- ☐ Support plan reference from new plans ("based on: X")

R12: Metaplans (NEW)

- ☐ Extract reusable template from completed plan
- ☐ Parameterize plan for different contexts
- ☐ Import metaplans from URLs or files
- ☐ Apply metaplan to current project
- ☐ Track metaplan lineage (which plans derived from which metaplan)
- ☐ Publish metaplans for community sharing

R13: Plan Forking (NEW)

- ☐ Fork step(s) to new plan with `riotplan fork`
- ☐ Automatic linking (original step blocked on forked plan)
- ☐ Fork during review ("this is too big")
- ☐ Fork during execution ("this should be reusable")
- ☐ Track fork relationships in dependency graph

R14: Embedded Version Tracking (NEW - Critical)

- ☐ Track revisions without requiring external Git repository
- ☐ Store diffs or snapshots within the plan/prompt structure
- ☐ Support linear timeline (v0.1 → v0.2 → v0.3)
- ☐ Enable "time travel" - render plan at any historical version
- ☐ Keep it lightweight (not a full DVCS)
- ☐ Human-readable history format

R15: Artifact Packaging (NEW)

- ☐ Package plan/prompt as single transportable file (`.riotplan`)
- ☐ Include all history, feedback, evidence in package
- ☐ Support "export current state only" mode (no history)
- ☐ Standardized format for interoperability
- ☐ Importable by tools and other users
- ☐ Unpack to standard directory structure

R16: RiotPrompt Integration (NEW)

- ☐ Apply feedback/revision patterns to prompts
- ☐ Share core infrastructure between riotplan and riotprompt
- ☐ Consider unified `@riotplan/core` package
- ☐ Prompt-specific: engine renders (Anthropic, OpenAI, etc.)
- ☐ Prompt-specific: template variable expansion

R17: Prompt Testing & Validation (NEW)

- ☐ Define expected behaviors for prompts
- ☐ Run prompts against test cases
- ☐ Compare output to expectations
- ☐ Generate automated feedback from test failures
- ☐ Support iterative improvement loops (prompt CI/CD)
- ☐ Track test results across revisions

R18: Attention Tracking (NEW)

- ☐ Track which plans/files the user is viewing over time
- ☐ Record session timeline: "morning on security plan, afternoon on protokoll"
- ☐ Surface recent context: "Before lunch you were working on Step 6"
- ☐ Identify relationships through attention patterns (viewed together = related)
- ☐ Support context recovery: "Where was I?" with timeline view

- ☐ Inform user proactively: "You context-switched 3 times today; here's the trail"

Rationale: When AI tools enable 100x productivity, context-switching becomes the bottleneck. The work isn't the problem—remembering what you were doing is. Attention tracking captures the "where was my head?" dimension that no file or plan can record on its own.

R19: Plan Contexts (NEW)

- ☐ Assign plans to contexts (work, personal, family, project-specific)
- ☐ Filter dashboard by context: `riotplan dashboard --context work`
- ☐ Support hierarchical contexts: `work/kjerneverk`, `personal/travel`
- ☐ Switch contexts easily: `riotplan context set personal`
- ☐ Default context per directory or config
- ☐ Cross-context visibility when needed: `riotplan dashboard --all`

Rationale: Life isn't just work. Someone might manage work projects, a ski trip, their kid's tutoring plan, and home renovation—all requiring structured planning. Contexts prevent work clutter from mixing with personal plans and vice versa, while keeping everything in one system.

R20: Document Rendering & Sharing (NEW)

- ☐ Render plan to PDF: `riotplan render --pdf`
- ☐ Render plan to HTML: `riotplan render --html`
- ☐ Render plan to DOCX: `riotplan render --docx`
- ☐ Render plan to clean Markdown (single file): `riotplan render --markdown`
- ☐ **Two rendering modes:**
  - ☐ `--full`: Everything—feedback records, revision history, evidence, changelog
  - ☐ `--current` (default for sharing): Just the plan as it is now, clean and focused
- ☐ Include/exclude specific sections: `--include feedback` or `--exclude evidence`
- ☐ Render at specific version: `--version v0.3`
- ☐ Custom templates for different audiences
- ☐ Generate shareable link or file: `riotplan share`
- ☐ Support "share for feedback" workflow with response capture

Rationale: Plans need to be shared, but different audiences need different views. Someone reviewing your plan for feedback wants the **current state**—clean, focused, no distractions. Someone doing a retrospective or audit wants **everything**—how it evolved, what was debated, what changed. Both should be one command away.

R21: Cross-Tool Content Capture (NEW)

- ☐ Import content from ChatGPT conversations (via export or paste)
- ☐ Preserve formatting/structure from AI UI outputs
- ☐ Capture images generated in external tools
- ☐ Import from clipboard with format detection
- ☐ Bridge between AI UIs and filesystem-based workflows
- ☐ Tag imported content with source (ChatGPT, Claude, etc.)

Rationale: Real workflows span multiple AI tools. ChatGPT Pro on mobile for brainstorming, Cursor for implementation, ChatGPT UI for image generation. Each context switch loses fidelity. RiotPlan should be the hub that captures and preserves content from wherever it originates.

R22: Cross-Project Awareness (NEW)

- ☐ Define an ecosystem of related projects (kjerneverk, fjellgrunn, utilarium, etc.)
- ☐ During plan creation, surface what capabilities exist elsewhere
- ☐ Identify when proposed features might belong in a different project
- ☐ Suggest: "This looks like it should be in project X, not here"
- ☐ Track which projects provide what capabilities (registry)
- ☐ Support "global context" that informs all plan creation in an ecosystem

Rationale: When revising kjerneverk for security, features were identified that belonged in other projects (offrecord, pressurelid, cardigantime, spotclean). Without ecosystem awareness, plans propose features in the wrong place. The system should know what exists across projects and guide features to the right home.

R23: Global Guidance & Style Configuration (NEW)

- ☐ Define personal/team rules that apply across all plans
- ☐ Configure writing style preferences (for blogs, documentation)
- ☐ Set coding conventions and patterns to prefer/avoid
- ☐ Provide default prompts and context for plan creation
- ☐ Support context-specific guidance (work vs personal style)
- ☐ Expose guidance to AI during plan creation and execution
- ☐ Hierarchical: global defaults → context overrides → plan overrides

Rationale: Everyone has preferences. "I always want TypeScript strict mode." "My blog posts should be conversational, avoid jargon." "Never suggest adding dependencies without justification." These rules shouldn't be re-stated in every plan—they should be ambient configuration that RiotPlan applies automatically.

R24: External Critical Review (NEW)

- ☐ Render plan to portable format (PDF, HTML) for external review
- ☐ Support sending to different AI models for "devil's advocate" analysis
- ☐ Ship with standard review prompts:
  - `devils-advocate.md`: "Critique this plan, identify weaknesses, find what's missing"
  - `research-grounding.md`: "Ground the key concepts in relevant research, cite papers"
  - `feasibility-check.md`: "Assess technical feasibility and implementation risks"
  - `audience-clarity.md`: "Review from perspective of [X audience], identify confusion"
  - `scope-creep.md`: "Identify where scope may be expanding beyond core goals"
- ☐ Import review results back into plan's feedback directory
- ☐ Tag imported reviews with source model and prompt type
- ☐ Support both manual workflow (export → external AI → import) and API integration
- ☐ Capture the clarifying conversation (questions asked, context provided)

Rationale: Different AI models have different strengths, training, and perspectives. Sending a plan to a "heavy" research-capable model (like ChatGPT Pro with Deep Research) for critical review surfaces blind spots, grounds concepts in academic literature, and provides a deliberate devil's advocate perspective. This multi-AI workflow is itself valuable deliberation that should be captured.

Example workflow:

```
# Render for external review
$ riotplan render pdf --for-review

# After receiving review from external AI, import it
$ riotplan review import ./path/to/review.docx \
    --source "ChatGPT 5.2 Pro Deep Research" \
    --prompt-type devils-advocate

# Or use integrated review (if API access configured)
$ riotplan review request --model "gpt-5-deep-research" --prompt devils-advocate
```

Research references identified through this pattern: - He et al. (2025) - "Plan-Then-Execute: User Trust and Team Performance with LLM Agents" (arXiv 2502.01390) - Tankelevitch et al. (2023) - Metacognitive challenges in GenAI workflows - Chirayath & Premamalini (2025) - "Cognitive offloading or overload?" - Design rationale literature (IBIS, ADRs)

R25: History Lifecycle Management (NEW)

- ☐ Commit to Git before any cleanup operation (safety net)
- ☐ Prune intermediate versions while keeping bookends (first + latest)
- ☐ Generate decision summary from feedback records before pruning
- ☐ Create cleanup manifest documenting what was removed and why
- ☐ Support "milestone" concept for explicit cleanup points
- ☐ Protect specific feedback records from cleanup (mark as "key decision")
- ☐ Allow selective version retention: `--keep-versions 1,5,12`
- ☐ Dry-run mode to preview cleanup effects

Rationale: Captured deliberation can become unwieldy—15+ feedback records and 10+ versions make it hard to find the signal in the noise. But the solution isn't to capture less; it's to provide lifecycle management. At milestones (when clarity is reached), users can "clean up" to focus on key versions while Git preserves the full history as a safety net.

The pattern: Compress, Don't Delete

```
Before cleanup:
  feedback/
    001-initial-review.md
    002-scope-discussion.md
    ... (15 records)
    015-final-refinements.md
  .history/
    v0.1/ v0.2/ ... v0.15/

After cleanup (at milestone):
  feedback/
    SUMMARY-key-decisions.md      # Consolidated key decisions
    015-final-refinements.md      # Most recent retained
  .history/
    v0.1/                         # Initial state (bookend)
    v0.15/                        # Final state (bookend)
```

```
cleanup-manifest.json          # What was pruned and why
```

Git remains the ultimate archive. Cleanup is explicit, not automatic.

R26: Extensible Execution Providers (NEW)

- ☐ Define execution provider interface for pluggable backends
- ☐ Step schema includes `executor` field specifying provider
- ☐ Built-in providers: anthropic, openai, gemini, azure, gcp, human, cicd
- ☐ Human provider for steps requiring manual completion
- ☐ Fallback configuration when provider unavailable
- ☐ Provider-specific configuration (API keys, models, etc.)
- ☐ Override executor at runtime: `riotplan execute 4 --executor human`
- ☐ Connection to existing kjerneverk execution packages

Rationale: Plans aren't executed by one tool. Different steps need different capabilities:

Step Type	Best Executed By
Code generation	Claude Code, Codex
Fact checking	OpenAI Deep Research
Human approval	Human
CI/CD pipeline	GitHub Actions
Research	Perplexity, Deep Research

The plan model should not be tied to one tool or platform.

Step schema with executor:

```typescript
interface PlanStep {
  title: string;
  executor: {
    provider: 'anthropic' | 'openai' | 'gemini' | 'human' | 'cicd' | 'custom';
    model?: string;  // e.g., "claude-code", "gpt-4"
    fallback?: 'human' | 'skip' | 'block';
    reason?: string; // Why this provider for this step
  };
  // ...
}
```

Example mixed-executor plan:

```markdown
### Step 2: Research Patterns
- **Executor**: openai:gpt-4-turbo (deep research)
- **Reason**: Needs web search and synthesis

### Step 4: Implement API
- **Executor**: anthropic:claude-code
- **Fallback**: human

### Step 6: Code Review
```

```
- **Executor**: human
- **Reason**: Requires human judgment
```

> **Plans define what to do. Providers define how to do it. Humans are a valid provider.**

R27: Completion Retrospectives and Self-Reflection (NEW)

- ☐ Generate `RETROSPECTIVE.md` on plan completion
- ☐ Generate `OUTCOME.md` summarizing what was delivered
- ☐ Phase-by-phase reflection: what worked, what was difficult
- ☐ Overall plan reflection: what would we do differently?
- ☐ Time estimate vs. actual tracking
- ☐ Key decisions extracted from feedback/ records
- ☐ Recommendations for similar future plans
- ☐ AI self-reflection when AI executes steps (confidence, difficulty)
- ☐ `riotplan complete --reflect` for interactive retrospective
- ☐ `riotplan complete --report pdf` for shareable report
- ☐ Insight extraction: mine feedback records for patterns

Rationale: Plans shouldn't just end—they should teach. Completion is an opportunity to learn:

1. **What actually happened** vs. what was planned
2. **What worked well** that should be repeated
3. **What was difficult** that future plans should avoid
4. **How estimates compared** to reality
5. **Unexpected discoveries** made during execution

New completion artifacts:

```
my-plan/
   RETROSPECTIVE.md      # Phase-by-phase + overall reflection
   OUTCOME.md            # What was delivered, artifacts produced
   retrospective.pdf     # Optional shareable report
```

RETROSPECTIVE.md structure:

```
# Retrospective: my-plan

## Overview
- Started: 2026-01-05 | Completed: 2026-01-14
- Duration: 9 days (estimated: 5 days)

## Phase Reflections
### Phase 1: Research
- What worked: Using deep research for patterns
- What was difficult: Scope kept expanding
- Key decision: Split into separate libraries

## Overall Reflection
### What Would We Do Differently?
```

```
1. Start with library extraction in mind
2. Add integration verification earlier

### Recommendations for Similar Plans
- Security utilities → extract to shared libraries
- Use "spawn project" pattern when scope grows

## Key Decisions Made
*Extracted from feedback/ records*
```

AI Self-Reflection (when AI executes steps):

```
### Step 4: Implement API
**Executor**: anthropic:claude-code
**AI Reflection:**
- 3 iterations needed to satisfy type checker
- Switched from class-based to functional after feedback
**Confidence**: High (92%) | **Difficulty**: Medium
```

> **Every completion is an opportunity to learn. Retrospectives turn deliberation into actionable insights.**

R28: Starter Templates and Wizards (NEW)

- ☐ Ship built-in templates for common scenarios
- ☐ `riotplan plan init <name> --template <template>` to use template
- ☐ `riotplan plan init <name> --wizard` for interactive guided creation
- ☐ Template management: list, show, save, export, import
- ☐ Users can create and share custom templates
- ☐ Templates encode best practices; metaplans are battle-tested refinements

Rationale: Don't make users start from scratch. New users face a blank slate and don't know what a "good" plan looks like.

Built-in templates:

Template	Use Case	Phases
`feature`	New feature	Research → Design → Implement → Test → Deploy
`bugfix`	Bug fix	Reproduce → Investigate → Fix → Verify
`refactor`	Refactoring	Audit → Plan → Execute → Verify
`security`	Security work	Assess → Identify → Remediate → Verify
`migration`	Migration	Analyze → Plan → Execute → Validate → Cleanup

Template	Use Case	Phases
`research`	Research	Question → Gather → Analyze → Synthesize
`content`	Blog/docs	Brainstorm → Outline → Draft → Edit → Publish
`blank`	Empty (default)	No pre-defined structure

Interactive wizard:

```
$ riotplan plan init my-feature --wizard

What type of work is this?
[1] Feature development
[2] Bug fix
[3] Content creation
...

How complex? [1] Small  [2] Medium  [3] Large

Who executes? [1] Mostly AI  [2] Mostly human  [3] Mixed
```

Template management:

```
riotplan template list
riotplan template show <template>
riotplan template save <name> --from <plan>
riotplan template export <name> > file.riottemplate
riotplan template import <file>
```

> **Ship templates that encode best practices. Templates are starting points; metaplans are battle-tested.**

Product justification: The "blank slate problem" kills adoption. New users staring at empty files don't know what a good plan looks like. Templates: - Lower barrier to entry (start immediately) - Encode best practices (learn by example) - Reduce decision fatigue (structure is provided) - Demonstrate value ("oh, this is what RiotPlan does")

Minimum viable implementation: Even just a folder of example plans works. Templates are **Phase 1 priority**, not a future nice-to-have.

CLI Requirements

```
# Plan Management
riotplan init <name>              # Create new plan
riotplan load <path>              # Load existing plan
riotplan validate <path>          # Validate structure
riotplan archive <path>           # Archive plan

# Status Operations
riotplan status [path]            # Show current status
```

```
riotplan list [path]              # List steps with status
riotplan blockers [path]          # Show blockers
riotplan dashboard                # Global multi-plan view

# Step Operations
riotplan add-step <title>         # Add new step
riotplan insert-step <n> <title>  # Insert at position
riotplan remove-step <n>          # Remove step
riotplan block <n> --reason "X"   # Mark step blocked
riotplan unblock <n>              # Mark step ready

# Execution
riotplan execute [step]           # Execute step(s)
riotplan resume                   # Resume from last state
riotplan skip <step>              # Skip step with reason
riotplan retry <step>             # Retry failed step

# Cross-Plan
riotplan link <path> --blocker-for <step>  # Link external plan
riotplan spawn <step> --to <path>          # Spawn new plan from step
riotplan sync                              # Sync linked plan statuses

# Feedback & Deliberation (NEW - Critical)
riotplan feedback <title>         # Start new feedback record
riotplan feedback --context "file:lines"   # Record with file context
riotplan feedback --proposed "text"        # What was proposed
riotplan feedback --response "text"        # What feedback was given
riotplan feedback --resolution "text"      # How it was resolved
riotplan feedback --commit                 # Commit after feedback

# Version Control (NEW)
riotplan commit [message]         # Commit current state
riotplan tag <version>            # Tag a plan version
riotplan history                  # Show revision history
riotplan diff [version]           # Show changes since version
riotplan changelog                # Generate/update CHANGELOG.md

# Archive & Knowledge Base (NEW)
riotplan archive <path>           # Move completed plan to archive
riotplan search <query>           # Search across all plans
riotplan related [path]           # Find related plans
riotplan categorize <path> <cat>  # Add category to plan
riotplan browse                   # Interactive plan browser

# Metaplans (NEW)
riotplan extract-template <path>  # Create metaplan from completed plan
riotplan import <url-or-path>     # Import a metaplan
riotplan apply <metaplan> [opts]  # Apply metaplan to current project
```

```
riotplan publish <path>           # Publish metaplan for sharing

# Plan Forking (NEW)
riotplan fork <step> --to <path>  # Fork step(s) to new plan
riotplan fork --reason "..."      # Document why forking
riotplan lineage [path]           # Show fork/spawn relationships

# Embedded Version Tracking (NEW)
riotplan revision                 # Show current revision
riotplan revisions                # List all revisions
riotplan revision <version>       # Show plan at specific revision
riotplan save [message]           # Save current state as new revision
riotplan restore <version>        # Restore to previous revision

# Artifact Packaging (NEW)
riotplan pack [path]              # Package plan as .riotplan file
riotplan pack --no-history        # Package current state only
riotplan unpack <file>            # Unpack .riotplan to directory
riotplan export <format>          # Export to other formats

# Attention Tracking (NEW)
riotplan timeline                 # Show today's attention timeline
riotplan timeline --week          # Show this week's timeline
riotplan wherewasi                # "Where was I?" - recent context
riotplan trail                    # Show the path through plans today

# Contexts (NEW)
riotplan contexts                 # List all contexts
riotplan context set <name>       # Switch to context
riotplan context add <name>       # Create new context
riotplan dashboard --context work # Filter by context
riotplan dashboard --all          # Show all contexts
riotplan assign <plan> <context>  # Assign plan to context

# Rendering & Sharing (NEW)
riotplan render --pdf             # Render current state to PDF
riotplan render --pdf --full      # Render everything (feedback, history, evidence)
riotplan render --html            # Render to HTML
riotplan render --docx            # Render to Word
riotplan render --markdown        # Render to single Markdown file
riotplan render --version v0.3    # Render plan as it was at v0.3
riotplan render --include feedback  # Current + specific sections
riotplan render --exclude steps   # Full minus specific sections
riotplan share                    # Generate shareable artifact (current state)
riotplan share --full             # Share with complete history
riotplan share --link             # Generate shareable link (if hosted)

# Cross-Tool Import (NEW)
```

```
riotplan import clipboard          # Import from clipboard
riotplan import chatgpt <file>     # Import ChatGPT export
riotplan import image <path>       # Import image into plan
riotplan capture                   # Interactive capture from AI UI

# Cross-Project Awareness (NEW)
riotplan ecosystem                 # Show all projects in ecosystem
riotplan ecosystem add <path>      # Add project to ecosystem
riotplan ecosystem capabilities    # List what each project provides
riotplan suggest-project <feature>  # Where should this feature live?
riotplan create --ecosystem        # Create plan with ecosystem awareness

# Global Guidance (NEW)
riotplan config                    # Show current configuration
riotplan config set <key> <value> # Set global preference
riotplan config guidance           # Show all guidance rules
riotplan config guidance add       # Add guidance rule
riotplan config style <context>    # Show style for context (blog, code, etc.)
riotplan config import <file>      # Import guidance from file

# External Critical Review (NEW)
riotplan render pdf --for-review  # Render plan for external review
riotplan review import <file>      # Import external review
riotplan review import <file> --source "ChatGPT 5.2 Pro" --prompt-type devils-advocate
riotplan review request            # Request review via API (if configured)
riotplan review prompts            # List available review prompts
riotplan review prompts show <name>  # Show specific review prompt

# History Lifecycle Management (NEW)
riotplan cleanup                   # Interactive cleanup wizard
riotplan cleanup --commit          # Commit to Git first, then cleanup
riotplan cleanup --summarize       # Generate decision summary before pruning
riotplan cleanup --keep-versions 1,8,15  # Keep specific versions
riotplan cleanup --dry-run         # Preview what would be removed
riotplan feedback summarize        # Generate summary of all feedback records
riotplan feedback protect <id>     # Mark feedback as "key decision" (won't be pruned)
riotplan milestone create <name>  # Mark current state as a milestone
riotplan milestone list            # Show all milestones
```

Part 6: Proposed Architecture

Core Types (already defined in riotplan/src/types.ts)

The existing type definitions are well-aligned with observations:

- `Plan` - Complete plan with metadata, files, steps, state
- `PlanStep` - Individual step with status, dependencies, timing

- `PlanPhase` - Optional grouping of steps
- `PlanState` - Execution state with blockers, issues, progress
- `Blocker` / `Issue` - Problems preventing progress
- `StatusDocument` - Schema for STATUS.md parsing

Proposed Module Structure

```
riotplan/
  src/
      index.ts              # Public API exports
      types.ts              # Type definitions (exists)

      plan/                 # Plan operations
          loader.ts         # Load plan from directory
          creator.ts        # Create new plan
          validator.ts      # Validate plan structure
          archiver.ts       # Archive completed plans

      status/               # Status tracking
          parser.ts         # Parse STATUS.md
          generator.ts      # Generate STATUS.md
          updater.ts        # Update status

      steps/                # Step operations
          numbering.ts      # Step renumbering
          operations.ts     # Insert, remove, move
          execution.ts      # Step execution

      dependencies/         # Dependency management
          internal.ts       # Within-plan dependencies
          external.ts       # Cross-plan dependencies
          graph.ts          # Dependency visualization

      execution/            # Execution engine
          executor.ts       # Execute steps
          quality.ts        # Quality gates
          resume.ts         # Resume interrupted

      dashboard/            # Multi-plan management
          registry.ts       # Plan registry
          status.ts         # Global status
          context.ts        # User context/attention
```

Key Design Decisions

1. File-Based State Plans are directories with markdown files. RiotPlan reads/writes these files rather than maintaining separate state.

Rationale: Plans are human-readable, version-controllable, and work without riotplan installed.

2. Progressive Enhancement RiotPlan adds capabilities without requiring changes to existing plans.

Rationale: Users already have plans; migration should be optional.

3. Cross-Repository Awareness RiotPlan can track plans across multiple repositories via a registry file.

Rationale: Complex work spans multiple projects (kjerneverk, fjellgrunn, utilarium).

4. Execution is Optional RiotPlan can manage plans without executing them (humans/AI agents execute).

Rationale: Execution is often done by AI agents (Cursor, Claude) not riotplan itself.

Part 7: Project Structure (NEW)

The scope of RiotPlan has grown significantly. Rather than cramming everything into two packages (`riotplan` and `riotplan-cli`), we should design for modularity from the start.

The Problem with a Single CLI

Looking at the CLI commands section, we have: - Plan management (init, load, validate, archive) - Status operations (status, list, blockers, dashboard) - Step operations (add-step, insert-step, remove-step, block, unblock) - Execution (execute, resume, skip, retry) - Cross-plan coordination (link, spawn, sync, fork, lineage) - Feedback capture (feedback with many options) - Version control (commit, tag, history, diff, changelog) - Revision tracking (revision, revisions, save, restore) - Artifact packaging (pack, unpack, export) - Attention tracking (timeline, wherewasi, trail) - Contexts (contexts, context set/add, assign) - Rendering & sharing (render, share) - Import/capture (import, capture) - Ecosystem awareness (ecosystem) - Configuration (config)

That's **60+ commands** across 15+ categories. A flat CLI becomes unwieldy. Users can't discover commands. Help text becomes a wall of text.

Proposed Project Structure

```
riotplan/                          # Monorepo root
  packages/
    core/                         # @riotplan/core
      src/
        types/                   # Type definitions
        plan/                    # Plan loading, creation, validation
        status/                  # Status parsing, generation
        steps/                   # Step operations
        feedback/                # Feedback capture
        history/                 # Revision tracking
        artifacts/               # Packaging (pack/unpack)
        registry/                # Plan registry, ecosystem
      package.json
```

```
cli/                          # @riotplan/cli
    src/
        commands/             # Command implementations
            plan/             # plan init, validate, archive
            status/           # status, list, dashboard
            step/             # step add, remove, block
            exec/             # exec run, resume, skip
            feedback/         # feedback create, list, show
            revision/         # revision save, restore, list
            artifact/         # artifact pack, unpack, share
            context/          # context set, list, assign
            render/           # render pdf, html, docx
            ecosystem/        # ecosystem add, capabilities
            config/           # config set, guidance
        cli.ts                # Main CLI entry point
    package.json

vscode/                       # @riotplan/vscode
    src/
        extension.ts          # VS Code extension entry
        providers/            # Tree views, decorations
        commands/             # VS Code command palette
        webviews/             # Embedded UI panels
        language/             # STATUS.md language support
    package.json

dashboard/                    # @riotplan/dashboard
    src/
        app/                  # React application
        components/           # UI components
            PlanTree/         # Plan hierarchy view
            Timeline/         # Attention timeline
            FeedbackPanel/    # Feedback viewer/editor
            StatusBoard/      # Kanban-style status
            EcosystemMap/     # Project relationships
        pages/                # Dashboard pages
    package.json

server/                       # @riotplan/server
    src/
        api/                  # REST/GraphQL API
        sync/                 # Cross-device sync
        attention/            # Attention tracking service
        share/                # Sharing/link generation
    package.json

renderer/                     # @riotplan/renderer
```

```
        src/
            pdf/                # PDF generation
            html/               # HTML generation
            docx/               # Word generation
            templates/          # Rendering templates
        package.json

apps/
    web/                        # Hosted dashboard (optional)
        ...

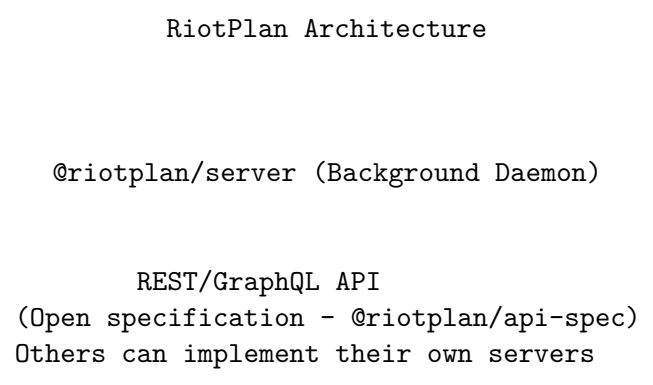
package.json                    # Monorepo root (pnpm/npm workspaces)
```

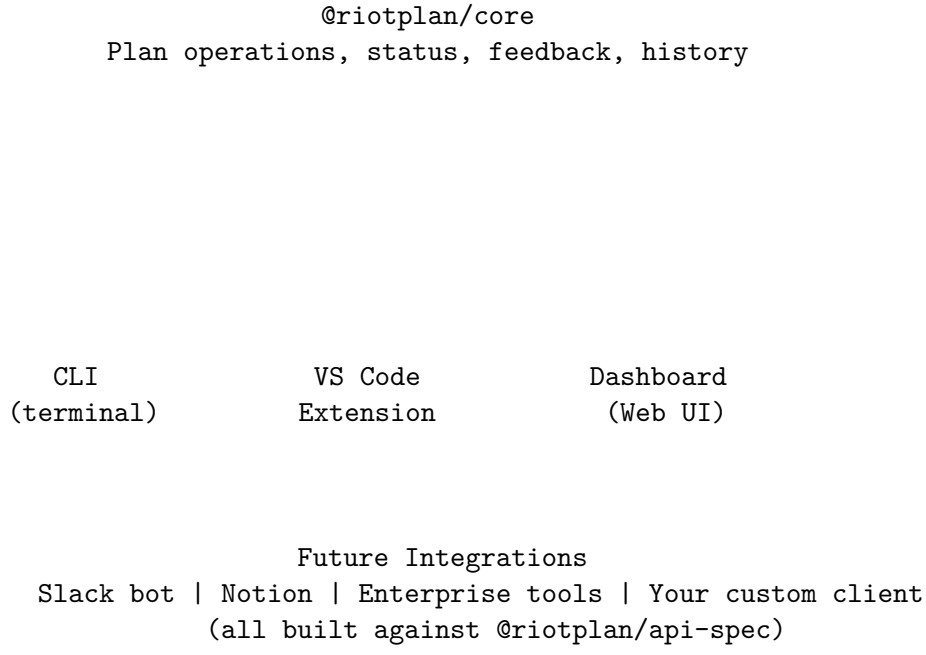
Package Responsibilities

Package	Purpose	Dependencies
`@riotplan/core`	Type definitions, plan operations, state management	None (leaf)
`@riotplan/cli`	Command-line interface	core, renderer, (optional: server client)
`@riotplan/vscode`	VS Code extension	core, (optional: server client)
`@riotplan/dashboard`	React-based local dashboard	server
`@riotplan/server`	**Background daemon** - REST/GraphQL API, attention tracking	core
`@riotplan/renderer`	PDF/HTML/DOCX generation	core
`@riotplan/api-spec`	**OpenAPI/GraphQL specification** (open for others to implement)	None

API-First Architecture (NEW)

The CLI isn't the end point—it's the first client. The architecture is **API-first**:





Key design decisions:

1. **Background daemon**: `@riotplan/server` runs locally, provides API + web dashboard
2. **Open API specification**: `@riotplan/api-spec` is a published spec others can implement
3. **CLI works both ways**: Directly against `core` (offline) or via API (when server running)
4. **All clients are equal**: CLI, VS Code, Dashboard are just different API consumers
5. **No SaaS dependency**: Everything runs locally, user owns their data
6. **Ecosystem-enabling**: Companies can build Slack bots, Notion integrations, enterprise versions

Why API-first matters:

Approach	Limitation
CLI-only	Non-developers excluded until GUI exists
GUI-first	Power users frustrated by clicking
API-first	**Both work.** CLI for power users, GUI for others, API for integrations

The open specification enables ecosystem: - Reference implementation: `@riotplan/server` - Companies can implement their own servers - Integrations can be built without forking - Enterprise features can be added without upstream changes

Project Philosophy: Impact Over Revenue

> "I'm at the point in my career and where I am where I'm really just looking for this to become something people use. I'm not motivated by revenue or money at this point."

This shapes fundamental decisions:

Traditional OSS	RiotPlan Approach
Build it, hope for stars	Design explicitly for adoption
Single implementation	Open spec + reference implementation
Community contributions	Community **implementations**
Measure: GitHub stars	Measure: People using it daily

What this means: - Documentation is a first-class deliverable (enables adoption) - API spec is published (enables ecosystem) - No telemetry, no accounts (removes friction) - Everything works offline (no dependency on us) - MIT/Apache license (no legal barriers)

RiotPlan succeeds if people use it to manage their work—not if it generates revenue.

CLI Command Grouping

Instead of flat commands, use subcommands:

```
# Plan operations
riotplan plan init <name>
riotplan plan validate
riotplan plan archive

# Status operations
riotplan status                    # Default: show status
riotplan status list               # List all steps
riotplan status dashboard          # Full dashboard

# Step operations
riotplan step add <title>
riotplan step remove <n>
riotplan step block <n>

# Execution
riotplan exec run [step]
riotplan exec resume
riotplan exec skip <step>

# Feedback
riotplan feedback create <title>
riotplan feedback list
riotplan feedback show <id>

# Revision tracking
```

```
riotplan revision save [message]
riotplan revision list
riotplan revision restore <version>

# Artifacts
riotplan artifact pack
riotplan artifact unpack <file>
riotplan artifact share

# Context
riotplan context set <name>
riotplan context list

# Render
riotplan render pdf
riotplan render html
riotplan render docx

# Ecosystem
riotplan ecosystem add <path>
riotplan ecosystem capabilities

# Config
riotplan config set <key> <value>
riotplan config guidance
```

Benefits of subcommand structure: - Discoverable: `riotplan feedback --help` shows all feedback commands - Organized: Related commands grouped together - Extensible: New command groups don't bloat the top level - Familiar: Follows patterns from git, docker, kubectl

Lifecycle-Aware Commands (NEW)

The CLI shouldn't just be utilities—it should understand **where you are in the plan lifecycle** and offer appropriate support.

Command	Phase	What It Does
`riotplan inception`	0	Start evidence gathering, explore the problem
`riotplan plan init`	1	Create plan structure from evidence
`riotplan review`	2	**Initiate feedback capture session**
`riotplan revise`	2	Apply feedback, create new version
`riotplan execute`	3	Run steps, **auto-update STATUS.md**
`riotplan verify`	4	Check integration, run quality gates
`riotplan complete`	5	Mark done, generate summary
`riotplan archive`	6	Move to historical reference

The key insight: `riotplan review` initiates feedback capture:

```
$ riotplan review


   REVIEW SESSION: my-feature (v0.3)

   What would you like to review?
   [1] Full plan overview
   [2] Specific step
   [3] Dependencies and blockers
   Or type feedback to start capturing...


> Step 3 scope is too broad

Creating feedback record: feedback/004-step3-scope.md
What's the concern? (Ctrl+D to finish)
> Step 3 covers both API design and implementation. Split them.

Feedback captured. Apply this feedback now? [Y/n] y

Applying...
- Split Step 3 into 3a (API design) and 3b (implementation)
- Updated EXECUTION_PLAN.md
- Incremented version to v0.4
- Committed: "plan(my-feature): split step 3 per review feedback"
```

riotplan execute auto-updates status:

```
$ riotplan execute 3

Executing Step 3: API Design
Pre-conditions:   Steps 1-2 complete, no blockers

Running quality gates...
    Lint passed
    Tests passed

Step 3 complete.
Updating STATUS.md: Step 3   →
Committed: "feat(my-feature): complete step 3"

Next: Step 4. Continue? [Y/n]
```

This transforms RiotPlan from "a bunch of commands" to "a workflow guide."

VS Code Extension Features

The `@riotplan/vscode` extension provides:

Feature	Description
Plan Explorer	Tree view of all plans in workspace
Status Decorations	Step status icons in file tree
Inline Feedback	Capture feedback without leaving editor
Attention Tracking	Automatic tracking of viewed files
Quick Actions	Command palette integration
Webview Panels	Embedded dashboard, timeline view
Language Support	Syntax highlighting for STATUS.md
Hover Information	Plan status on hover over references

Dashboard Features

The `@riotplan/dashboard` provides a local web UI:

Feature	Description
Multi-Plan Overview	See all plans across contexts
Attention Timeline	Visual timeline of where you've been
Feedback Browser	Read/write feedback with rich UI
Status Board	Kanban-style step management
Ecosystem Map	Visualize project relationships
Blocker Graph	See what's blocking what
Search	Full-text search across plans
Render Preview	Preview rendered output before export

Start locally with:

```
riotplan dashboard serve
# Opens http://localhost:3847
```

Implementation Phases

Phase	Packages	Focus
1	`core`, `cli`	Basic plan operations, status tracking
2	`core`, `cli`	Feedback capture, revision tracking
3	`renderer`, `cli`	Document rendering (PDF, HTML, DOCX)
4	`vscode`	VS Code extension (basic)
5	`server`, `dashboard`	Local dashboard, attention tracking
6	`vscode`	Enhanced VS Code integration
7	All	Ecosystem awareness, cross-project

Why This Structure?

1. **Separation of concerns**: Core logic is reusable across CLI, VS Code, dashboard
2. **Independent deployment**: Can use CLI without VS Code, or VS Code without dashboard
3. **Testability**: Core can be tested in isolation

4. **Contribution-friendly**: Different teams/contributors can own different packages
5. **Incremental delivery**: Ship CLI first, add VS Code later
6. **Future-proof**: Web app, mobile app, other integrations use same core

Proposed Repository Setup (For Review)

Before creating a plan for implementation, we need to establish where the code will live. Here are two options:

Option A: Monorepo (Recommended) All packages in a single repository with workspace management:

```
riotplan/                          # Single GitHub repo
  .github/
    workflows/                   # CI/CD for all packages
  packages/
    core/                        # @riotplan/core
    cli/                         # @riotplan/cli
    vscode/                      # @riotplan/vscode
    dashboard/                   # @riotplan/dashboard
    server/                      # @riotplan/server
    renderer/                    # @riotplan/renderer
  apps/
    web/                         # Optional hosted dashboard
  docs/                          # Documentation site
  examples/                      # Example plans
  package.json                   # Workspace root
  pnpm-workspace.yaml            # Or npm/yarn workspaces
  turbo.json                     # Or nx.json for build orchestration
```

Pros: Atomic commits across packages, easier refactoring, single CI/CD **Cons**: Larger repo, all-or-nothing cloning

Option B: Multi-Repo Separate repositories for each package:

```
GitHub Organization: riotplan/
  riotplan-core/                 # @riotplan/core
  riotplan-cli/                  # @riotplan/cli
  riotplan-vscode/               # @riotplan/vscode
  riotplan-dashboard/            # @riotplan/dashboard
  riotplan-server/               # @riotplan/server
  riotplan-renderer/             # @riotplan/renderer
  riotplan-docs/                 # Documentation
```

Pros: Independent versioning, smaller clones, clear ownership **Cons**: Cross-repo changes are harder, version coordination overhead

Recommendation Start with Monorepo (Option A) because: - Early development benefits from atomic changes - Easier to refactor interfaces between packages - Single place for issues and discussions - Can split later if needed (monorepo → multi-repo is easier than reverse)

Proposed Directory Creation

Action Required: Before implementing, create the following structure:

```
# Create the monorepo
mkdir -p ~/gitw/riotplan
cd ~/gitw/riotplan

# Initialize
git init
npm init -y  # Or pnpm init

# Create package directories
mkdir -p packages/{core,cli,vscode,dashboard,server,renderer}/src
mkdir -p apps/web
mkdir -p docs
mkdir -p examples

# Create workspace config (for pnpm)
cat > pnpm-workspace.yaml << 'EOF'
packages:
  - 'packages/*'
  - 'apps/*'
EOF

# Create root package.json
cat > package.json << 'EOF'
{
  "name": "riotplan",
  "private": true,
  "workspaces": ["packages/*", "apps/*"],
  "scripts": {
    "build": "turbo run build",
    "test": "turbo run test",
    "lint": "turbo run lint"
  },
  "devDependencies": {
    "turbo": "^2.0.0",
    "typescript": "^5.0.0"
  }
}
EOF

# Initialize each package
for pkg in core cli vscode dashboard server renderer; do
  cat > packages/$pkg/package.json << EOF
{
  "name": "@riotplan/$pkg",
```

```
  "version": "0.0.1",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "scripts": {
    "build": "tsc",
    "test": "vitest"
  }
}
EOF
done
```

Package Dependencies (Initial)

```
@riotplan/core          # No dependencies (leaf package)
       ↑
          @riotplan/cli         # Depends on core, renderer
          @riotplan/vscode      # Depends on core
          @riotplan/dashboard   # Depends on core
          @riotplan/server      # Depends on core
          @riotplan/renderer    # Depends on core
```

Next Steps

1. **You**: Create the GitHub repository and directory structure
2. **You**: Review and adjust package names, structure
3. **Then**: Create implementation plan for Phase 1 (`core` + `cli`)
4. **Then**: Execute Phase 1 implementation

The plan for implementing RiotPlan will be **created using the patterns defined in this analysis**—which means it will have its own: - `evidence/` (this analysis document) - `feedback/` (the 6 feedback records from this conversation) - `SUMMARY.md`, `EXECUTION_PLAN.md`, `STATUS.md` - Numbered step files

The first real test of RiotPlan will be building RiotPlan itself.

Documentation Requirements (Critical)

RiotPlan should launch with comprehensive documentation that enables others to use it. This isn't an afterthought—it's a core deliverable.

Audience-Aware Documentation (NEW) Not everyone needs the philosophy. Documentation should serve **two audiences**:

```
                        LANDING PAGE

    "RiotPlan helps you manage complex, multi-step work."

    [Get Started →]                    [Why RiotPlan? →]
    (practical path)                   (philosophical path)
```

```
PRACTICAL PATH                  PHILOSOPHICAL PATH
"Just show me how"              "Help me understand why"

- Quick Start (5 min)           - Philosophy & Motivation
- Basic Commands                - Human Stories
- Common Workflows              - Research Grounding
- Examples                      - Design Decisions
                                - "The Book" (full guide)
No jargon.                      All the jargon.
No philosophy.                  All the philosophy.
Just tools.                     The full picture.
```

Key principle: Simple first, deep optional.

Path	Tone	Example
Practical	Direct, instructional	"Run this command. You'll see this output."
Philosophical	Reflective, explanatory	"The insight behind this feature is…"
Reference	Neutral, comprehensive	"Parameters: –format (pdf|html|docx)"

Documentation Hierarchy (5 Levels)

```
LEVEL 1: LANDING PAGE (30 seconds)
  "RiotPlan helps you plan, track, and remember why."
  One-liner + 3 bullet benefits + "Get Started" button

LEVEL 2: QUICK START (5 minutes)
  Install → Create first plan → See it work
  No philosophy, just action

LEVEL 3: TUTORIALS (30 minutes)
  Walk through real scenarios:
  - Content creator planning a tutorial series
  - Developer building a feature
  - Team planning a migration

LEVEL 4: CONCEPTS (as needed)
  Explain plan structure, feedback, phases, etc.
  Reference when stuck, not required reading

LEVEL 5: PHILOSOPHY (for the curious)
  The full story, research references
  Completely optional
```

Real Scenario Walkthroughs Show, don't tell. Include these case studies:

Scenario 1: Content Creator > Maya plans a 10-episode tutorial series with an AI assistant. She outlines episodes (plan creation), discusses structure with AI (feedback captured), revises based on editor feedback, and tracks writing progress. Six months later, creating Season 2, she reviews her original reasoning.

Scenario 2: Developer > Alex builds a new API. Three months in, a new team member asks "why pagination instead of cursor-based?" Alex points to `feedback/003-pagination-decision.md` which captured the debate and tradeoffs.

Scenario 3: Team Migration > A team migrates from REST to GraphQL. The plan captures why GraphQL, the strategy, concerns and how addressed, and outcomes. Next year, another team learns from this.

Every entry point should: 1. Start simple: What does this do? How do I use it? 2. Link to depth: "Want to understand why? See Philosophy."

Documentation Structure

```
riotplan/
  docs/                          # Documentation site (GitHub Pages)
     public/                     # Static markdown content
        index.md                 # Landing page (both paths)

        quick-start/             # PRACTICAL PATH (no jargon)
           5-minute-intro.md     # Get running in 5 minutes
           basic-commands.md     # Essential commands only
           common-workflows.md   # Copy-paste examples
           cheatsheet.md         # One-page reference

        philosophy/              # PHILOSOPHICAL PATH (full depth)
           motivation.md         # Why RiotPlan exists
           human-stories.md      # The 14-hour day story
           research.md           # Academic grounding
           design-decisions.md   # Why it works this way
           the-book/             # The full guide
               00-introduction.md
               01-what-is-riotplan.md
               02-core-concepts.md
               03-your-first-plan.md
               04-feedback-and-revision.md
               05-execution.md
               06-working-with-multiple-plans.md
               07-contexts-and-ecosystems.md
               08-advanced-patterns.md

        reference/               # FOR EVERYONE
           cli-commands.md
           file-formats.md
```

```
        configuration.md
        api/

    guides/                 # How-To Guides
        capturing-feedback.md
        cross-project-dependencies.md
        creating-metaplans.md
        using-with-cursor.md
        using-with-chatgpt.md

    examples/               # Example Plans
        simple-feature/
        security-audit/
        blog-post/
        multi-project/

src/                         # React documentation site
    App.tsx
    components/
    ...
package.json
vite.config.ts
```

Key Documents 1. Philosophy & Motivation (`philosophy.md`)

This document answers: *Why does RiotPlan exist?*

Contents: - The problem of lost deliberation - Why conversations matter more than documents - The metacognitive approach to planning - Plans as evolving records of reasoning - Beyond code: plans for any structured work

Source material: The "Philosophical Foundation" section of this analysis.

2. Human Stories (`human-stories.md`)

This document answers: *What problem does this actually solve?*

Contents: - "A Day Without RiotPlan" - the 14-hour cognitive grind - "A Day With RiotPlan" - the streamlined workflow - Pain points table with real examples - Quotes and scenarios from actual experience

Source material: The "Human Experience" section of this analysis.

3. The Startup Guide ("The Book")

This is a comprehensive guide for new users. It should read like a book, not a reference manual.

Chapter	Title	What It Covers
00	Introduction	What you'll learn, who this is for
01	What is RiotPlan?	High-level overview, key value proposition
02	Core Concepts	Plans, steps, status, feedback, evidence, revision

Chapter	Title	What It Covers
03	Your First Plan	Step-by-step tutorial creating a simple plan
04	Feedback & Revision	How to capture deliberation, iterate on plans
05	Execution	Running plans, tracking progress, handling blockers
06	Multiple Plans	Dashboard, context switching, dependencies
07	Contexts & Ecosystems	Work vs personal, cross-project awareness
08	Advanced Patterns	Metaplans, forking, artifact packaging

Goal: Someone should be able to read this guide and understand not just *how* to use RiotPlan, but *why* it works the way it does.

Documentation Site (GitHub Pages) The documentation site should be: - **React-based** (like riotprompt docs) - **Markdown-driven** (content in `/public/*.md`) - **Searchable** (full-text search across all docs) - **Mobile-friendly** (responsive design) - **Versioned** (match documentation to releases)

Technology stack (matching riotprompt): - Vite for build - React for UI - Markdown rendering - GitHub Pages deployment

Documentation as Phase 0.5 Documentation should be created **before** Phase 1 implementation:

Phase	Focus
0	Analysis (this document)
0.5	**Documentation foundation** - philosophy, human stories, getting started outline
1	Core implementation
1.5	**CLI documentation** - update docs as features ship
2	Feedback & revision implementation
2.5	**Update getting started guide** with real examples
…	Continue pattern: ship code, update docs

Why documentation first?

1. Forces clarity on concepts before implementation
2. Serves as specification for what to build
3. Others can review/contribute to docs while code is being written
4. Marketing: “Coming soon” landing page with philosophy builds interest
5. Prevents documentation debt

Documentation Deliverables Checklist

- ☐ `philosophy.md` - Motivation and approach (extract from this analysis)
- ☐ `human-stories.md` - Before/after experience (extract from this analysis)
- ☐ Getting Started Guide - 8 chapters covering concepts to advanced use
- ☐ CLI Reference - All commands documented
- ☐ File Format Reference - STATUS.md, plan structure, feedback format
- ☐ Configuration Reference - Global guidance, contexts, ecosystem
- ☐ Example Plans - At least 4 complete examples
- ☐ GitHub Pages Site - React-based, deployed, searchable
- ☐ README.md - Quick overview linking to full docs

Part 8: Revision Tracking & Artifacts (NEW)

This section addresses a fundamental insight: **plans and prompts need version history without requiring Git repositories**.

The Problem with Git-Per-Plan

Git is powerful but creates friction:

Friction Point	Impact
New repo per plan	Overhead for small artifacts
Clone/push/pull ceremony	Slows iteration
GitHub account required	Barrier for casual use
Submodule complexity	If plans live in larger repos
Binary history	Not human-inspectable

For a single prompt that goes through 5 revisions, creating a Git repository is overkill.

Embedded Version Tracking

Plans and prompts should track their own history:

```
my-plan/
   SUMMARY.md                    # Current state
   EXECUTION_PLAN.md
   STATUS.md
   plan/
   feedback/
   evidence/
   .history/                     # NEW: embedded version tracking
       manifest.json             # Version metadata
       v0.1/                     # Snapshot or diffs
           snapshot.tar.gz       # Full state at v0.1
           meta.json             # Timestamp, author, message
       v0.2/
```

```
        changes.diff          # Diff from v0.1
        meta.json
    v0.3/
        changes.diff
        meta.json
```

Storage strategies:

Strategy	Pros	Cons
Full snapshots	Simple, self-contained	Storage-heavy
Unified diffs	Compact, standard format	Harder to render
JSON diffs	Structured, queryable	Custom format
Hybrid	Best of both	More complex

Recommended approach: Store full snapshot at v0.1, then diffs for subsequent versions. Optionally compress old snapshots.

The Artifact Format

A `.riotplan` file is a **packaged plan** that can be transported:

```
my-plan.riotplan  (zip archive)
  manifest.json
   {
     "format": "riotplan/1.0",
     "name": "secure-riotprompt",
     "version": "0.7",
     "created": "2026-01-10",
     "packed": "2026-01-14",
     "includes_history": true
   }
  current/                      # Current state
      SUMMARY.md
      EXECUTION_PLAN.md
      STATUS.md
      plan/
  history/                      # Version history
      v0.1/ ... v0.7/
  feedback/                     # Deliberation records
  evidence/                     # Inception materials
```

Export modes:

```
# Full artifact with history
riotplan pack my-plan/
# → my-plan.riotplan (includes everything)

# Current state only (smaller, no history)
riotplan pack my-plan/ --no-history
```

```
# → my-plan.riotplan (just current/)

# Specific version
riotplan pack my-plan/ --version v0.3
# → my-plan-v0.3.riotplan (state at v0.3)
```

RiotPrompt Integration

The same patterns apply to prompts:

```
my-prompt.riotprompt/
   prompt.md                    # Current prompt content
   manifest.json                # Metadata
   variables.json               # Template variables
   .history/                    # Revision history
   feedback/                    # How prompt evolved
   evidence/                    # What informed it
   tests/                       # Validation checks
       test-001.yaml
       test-002.yaml
   renders/                     # Pre-rendered outputs
       anthropic.json           # Claude format
       openai.json              # GPT format
       raw.txt                  # Plain text
```

Prompt testing example:

```yaml
# tests/test-001.yaml
name: "Code generation produces valid JavaScript"
description: "Verify the prompt generates syntactically correct code"

input:
  variables:
    language: "javascript"
    task: "sort an array"

run:
  model: "claude-sonnet-4-20250514"
  max_tokens: 500

assertions:
  - type: contains
    value: "function"
  - type: syntax_valid
    language: javascript
  - type: not_contains
    value: "I cannot"

on_failure:
```

```yaml
  action: create_feedback
  message: "Prompt failed code generation test"
```

Shared Core Infrastructure

Both RiotPlan and RiotPrompt need:

```
@riotplan/core/
  history/                    # Version tracking
    tracker.ts                # Save/load revisions
    diff.ts                   # Generate/apply diffs
    timeline.ts               # Linear history management
  artifact/                   # Packaging
    packer.ts                 # Create .riot* files
    unpacker.ts               # Extract .riot* files
    manifest.ts               # Metadata handling
  feedback/                   # Deliberation capture
    recorder.ts               # Create feedback records
    indexer.ts                # Search feedback
  evidence/                   # Inception materials
     cataloger.ts             # Organize evidence
```

Then: - `@riotplan/riotplan` uses core + plan-specific features - `@riotplan/riotprompt` (or existing `riotprompt`) uses core + prompt-specific features

The IDE Plugin Vision

Future integration:

```
  VS Code / Cursor


    Plan/Prompt Editor      History Panel

    # Summary               v0.7 (current)
    This plan...            v0.6 - Added evidence dir
                            v0.5 - Expanded lifecycle
    ## Goals                v0.4 - Human stories
    1. Security             v0.3 - Multi-modal
    2. Performance          [View v0.3]



    Feedback Panel
    [New Feedback] [Save Revision] [Pack Artifact]

    Feedback 006: Revision tracking discussion
    Feedback 005: Evidence directory
    ...
```

Why This Matters

Without artifact packaging: - Plans are tied to their directory location - Sharing requires zipping manually + losing structure - No standard format for consumption

With artifact packaging: - `riotplan pack my-plan/` → portable file - Email to colleague, they `riotplan unpack` - Include in blog post: "Download the plan here" - Metaplans are just artifact files

Without embedded versioning: - History requires Git - Each plan needs a repository - Or history is lost

With embedded versioning: - History travels with the plan - No external tools required - `riotplan revisions` shows history anywhere

---

Part 9: Goals for RiotPlan

Primary Goals

1. **Reduce Cognitive Overhead**: Users shouldn't have to remember "where was I?" or "what's blocked?"
2. **Standardize Plan Structure**: Consistent structure enables tooling and automation.
3. **Enable Multi-Session Work**: Plans persist state across sessions, days, and context switches.
4. **Support Plan Evolution**: Plans change; track revisions and support iteration.
5. **Coordinate Complex Work**: Handle cross-project dependencies and plan spawning.

What RiotPlan Is NOT

- **Not an LLM/AI system**: RiotPlan is tooling for plans, not an AI itself
- **Not an execution engine**: Plans are executed by humans or AI agents, not riotplan
- **Not a project manager**: RiotPlan tracks technical plans, not business workflows
- **Not a replacement for git**: Plans live in repositories; riotplan adds structure

A Note on Examples: Technical Bias

The examples throughout this document are heavily technical. The evidence files (`what-happened-*.md`) all describe creating Node.js libraries with interdependencies. The human experience story follows someone managing security implementations and version strategies. This creates a natural bias toward viewing RiotPlan as a tool for software development.

But consider: every time you write a blog post, you go through a similar process: - Gather evidence (research, references, quotes) - Consider the goal (what point are you making?) - Structure the approach (outline, sections, flow) - Draft and revise (multiple iterations) - Get feedback (from editors, colleagues, or yourself after sleeping on it) - Finalize and publish

That process isn't fundamentally different from creating a technical plan. The *output* is different—a blog post instead of a library—but the *process* of deliberation, structuring, and iteration is remarkably similar.

Beyond Code: RiotPlan as Metacognitive Tool

While the current implementation focuses on software development, RiotPlan's patterns apply to **any complex, structured work**:

Domain	Example Plan
Software	"Implement OAuth 2.0 authentication"
Documents	"Write technical specification for Feature X"
Blog Posts	"Draft article on emerging security practices"
Podcasts	"Develop script for episode on AI workflows"
Financial Models	"Build 5-year projection model"
Academic Papers	"Research and write paper on topic Y"
Business Strategy	"Develop market entry plan for Region Z"
Creative Projects	"Produce documentary about subject W"
Charters	"Define team charter and operating principles"

The core insight: **structured deliberation is universal**.

The Content Creation Connection

This analysis needs to acknowledge something important: the process of creating a blog entry, podcast script, or charter follows the same pattern:

1. **Evidence gathering**: Research, references, prior art, quotes, ideas
2. **Goal consideration**: What's the point? Who's the audience?
3. **Structure**: Outline, flow, key sections
4. **Drafting**: First attempt at the content
5. **Revision**: Multiple iterations based on feedback
6. **Finalization**: Polish and publish/deliver

Coming up with a plan for a programming task *is* different from planning a blog post in the specifics—one produces code, the other produces prose. But the *process* of gathering evidence, considering goals, structuring approach, revising based on feedback, and refining toward a final output is not dissimilar.

This suggests two possibilities:

1. **RiotPlan broadens**: The same tool supports both technical plans and content creation plans. A "plan" for a blog post would have evidence (research), feedback (editorial comments), revisions (drafts), and a final output (published post).
2. **A sister project emerges**: A separate tool (RiotContent? RiotDraft?) applies these patterns specifically to content creation, sharing the core infrastructure but with domain-specific features.

Either way, the patterns identified here—evidence capture, feedback preservation, revision tracking, artifact packaging—apply beyond code.

Plans Are Documents

Here's a deeper insight: **a plan is itself a document**.

When we talk about "executing a plan," we're really saying: "use this document to guide work." When we talk about "revising a plan," we're editing a document. When we "render a plan at a point in time," we're producing a document snapshot.

The manifestation of a plan—what you see when you open `SUMMARY.md` or `EXECUTION_PLAN.md`—is just the output of a documentation process. The plan *is* structured content that went through evidence gathering, drafting, revision, and feedback.

This reframing is important because it means: - RiotPlan's patterns for managing plans are *also* patterns for managing documents - A "content creation" variant wouldn't be fundamentally different—it would just have different templates and rendering targets - The line between "plan for creating something" and "the thing being created" blurs when the thing is itself a document

Consider this very analysis: it's a document that describes how to make plans. But it's also *itself* a plan-like artifact—it has evidence (`what-happened-*.md`), feedback (`feedback/*.md`), revisions (v0.1 through v0.7), and a rendered output (this markdown, and the PDF). The tool we're designing could manage the creation of the very document describing it.

RiotPlan is Really About...

> Capturing metacognitive ideas and input that shape execution—whether that execution produces code, content, or anything else.

This means: - **Thinking about thinking** - How should we approach this? - **Structured planning** - Breaking complex work into steps - **Deliberation preservation** - Recording the reasoning, not just decisions - **Knowledge accumulation** - Learning from past approaches - **Process reuse** - Applying how we planned, not just what we planned

While the initial implementation focuses on code (because that's the immediate pain point), the architecture should not *assume* code. Plans could be: - Code implementations - Document drafts - Blog posts and articles - Podcast scripts and show notes - Research agendas - Learning curricula - Project proposals - Team charters - Anything requiring structured multi-step effort with deliberation

Success Criteria

1. **Zero-Config Start**: `riotplan init my-feature` creates a valid plan immediately
2. **Context Recovery**: `riotplan status` tells you exactly where you are
3. **Blocker Visibility**: Blocked work is immediately obvious
4. **Cross-Plan Awareness**: Dependencies across projects are tracked
5. **Human-Readable**: Plans work without riotplan installed

Part 10: Implementation Roadmap

Phase 1: Foundation (v0.1.0)

Focus: Core plan management, status tracking, **and feedback capture**

- ☐ `loadPlan(path)` - Load plan from directory
- ☐ `createPlan(config)` - Create new plan
- ☐ `parseStatus(content)` - Parse STATUS.md
- ☐ `generateStatus(plan)` - Generate STATUS.md
- ☐ `validatePlan(path)` - Validate structure
- ☐ `createFeedback(context)` - Create feedback record (CRITICAL - day one feature)
- ☐ `commitPlan(message)` - Commit plan changes to git

CLI Commands: - `riotplan init <name>` - `riotplan status [path]` - `riotplan list [path]` - `riotplan validate [path]` - `riotplan feedback <title>` - Start feedback record - `riotplan commit [message]` - Commit changes

Rationale: Feedback capture is a Phase 1 feature, not Phase 4, because: 1. Every plan revision involves deliberation worth capturing 2. Without it from the start, early deliberation is lost 3. It's the philosophical foundation of the tool 4. It can be implemented simply (markdown files) before complex features

Phase 2: Step Operations (v0.2.0)

Focus: Step manipulation and dependencies

- ☐ Step insertion with auto-renumbering
- ☐ Step removal with auto-renumbering
- ☐ Dependency tracking
- ☐ Blocker management

CLI Commands: - `riotplan add-step <title>` - `riotplan insert-step <n> <title>` - `riotplan remove-step <n>` - `riotplan block <n> --reason` - `riotplan unblock <n>`

Phase 3: Execution Support (v0.3.0)

Focus: Execution tracking and quality gates

- ☐ Step execution tracking
- ☐ Quality gate definitions
- ☐ Resume from interruption
- ☐ Skip/retry operations

CLI Commands: - `riotplan execute [step]` - `riotplan resume` - `riotplan skip <step>` - `riotplan retry <step>`

Phase 4: Cross-Plan (v0.4.0)

Focus: Multi-plan and cross-repository coordination

- ☐ Plan registry
- ☐ External dependency linking
- ☐ Status synchronization
- ☐ Multi-plan dashboard

CLI Commands: - `riotplan dashboard` - `riotplan link <path>` - `riotplan spawn <step> --to <path>` - `riotplan sync`

Part 11: Research References (NEW)

RiotPlan's approach is grounded in academic research. These references support the design decisions and can be cited in documentation.

Academic Papers

Paper	Authors	Year	Key Contribution
Plan-Then-Execute: User Trust and Team Performance with LLM Agents	He et al.	2025	Involving humans in AI planning increases trust and outcomes; user-reviewed plans lead to better performance
Cognitive and Metacognitive Challenges in GenAI Design Workflows	Tankelevitch et al.	2023	Three metacognitive phases in AI work: (1) formulating the prompt, (2) evaluating output, (3) deciding how to integrate
Cognitive offloading or overload? How AI alters coping	Chirayath & Premamalini	2025	AI tools can reduce cognitive load or increase it; design must be careful
Metacognition in HCI: Designing for Planning and Flexibility	Conway & Zachry	2025	Metacognition is underexplored in tool design; RiotPlan addresses this gap
Exploring the Potential of Metacognitive Support Agents	Various	2024	AI tools that prompt users to reflect improve outcomes

Reference URLs

- **He et al. 2025**: arXiv 2502.01390, ResearchGate
- **Tankelevitch et al. 2023**: DIS 2023 / arXiv
- **Chirayath 2025**: Frontiers in Psychology
- **Design Rationale**: Wikipedia
- **ADRs**: Agile Alliance

Foundational Concepts

Concept	Origin	Relevance to RiotPlan
Design Rationale	Software Engineering	Capturing the "why" behind design decisions
Architecture Decision Records (ADRs)	Agile Alliance	Lightweight decision logs; industry validation
Issue-Based Information System (IBIS)	Rittel & Kunz, 1970	First framework for capturing design debates

Key Research Findings

1. **Involving humans in AI planning increases trust and outcomes** (He et al. 2025)
2. **Capturing reasoning is beneficial for future understanding** (Design Rationale literature)
3. **Tools that enhance reflective thinking improve performance** (Tankelevitch et al. 2023)
4. **Cognitive offloading helps, but can lead to overload if not managed** (Chirayath 2025)
5. **Design rationale tools historically failed due to being too intrusive** (Historical lesson—RiotPlan must be low-friction)

How to Use These References

- **Philosophy documentation**: Cite papers to ground concepts
- **Academic contexts**: Full citations available
- **Practical documentation**: Don't cite—just apply the principles
- **Implementation**: These findings inform design decisions (e.g., R25 cleanup addresses overload concern)

Part 12: Terminology Guidance (NEW)

The Problem with "Metacognitive"

The term "metacognitive" is: - Academically accurate - Grounded in research - Sounds like German philosophy - Buzzwordy to practitioners - Alienating to non-academic audiences

> "It's a bit too German philosophy for many." — User feedback

Alternative Terms

Instead of…	Use…	Context
"Metacognitive planning"	**Reflective planning**	General audience

Instead of…	Use…	Context
“Metacognitive process”	**Thinking about the approach**	Casual explanation
“Metacognitive data”	**Decision log** or **reasoning trace**	Technical docs
“Metacognitive framework”	**Planning and reflection tool**	Marketing/docs
“Metacognition”	**Thinking about thinking**	If you must explain it

Plain Language Equivalents

Jargon	Plain English
“Capture metacognitive deliberation”	“Save the conversations where you figured out how to approach the problem”
“Metacognitive phases”	“The times when you’re deciding *how* to do something, not just doing it”
“Metacognitive support”	“Help thinking through your approach”

When to Use “Metacognitive”

Use it: - In the Philosophy documentation (for those who want depth) - When citing research papers - Once, with explanation, then use plain terms after

Don’t use it: - Quick Start documentation - CLI help text - First-time user experience - Without explaining what it means

Example: Good vs Bad

Bad (buzzwordy): > “RiotPlan is a metacognitive framework for capturing deliberative processes in multi-session AI workflows.”

Good (plain): > “RiotPlan saves your planning conversations so you don’t lose the reasoning behind decisions.”

Good (with depth available): > “RiotPlan saves your planning conversations. [Why this matters →]”

Documentation Path Alignment

Path	Terminology Style
Quick Start	Plain English only. No jargon.
Practical Guides	Simple terms. Link to philosophy if relevant.
Philosophy	Academic terms OK, but always explained.
Reference	Technical terms, neutrally presented.

The One-Liner

> **Use "metacognitive" in the philosophy docs. Use "thinking about how to approach the problem" everywhere else.**

Part 13: Open Questions

Design Questions

1. **Plan Discovery**: How does riotplan find plans? Scan directories? Registry file? Config?
2. **Step Numbering**: Should steps always be 01, 02, 03 or allow gaps (01, 02, 05)?
3. **Phase Support**: Are phases required or optional? How are they defined?
4. **Execution Tracking**: Should riotplan track WHO executed a step (human vs AI)?
5. **Version Conflicts**: What happens if STATUS.md is edited while plan is being updated?

Integration Questions

1. **Git Integration**: Should riotplan create commits when updating STATUS.md?
2. **IDE Integration**: How should riotplan work with Cursor/VS Code?
3. **AI Agent Integration**: How do AI agents interact with riotplan?
4. **Notification System**: How are users notified when blockers resolve?

Scope Questions

1. **Template System**: Should riotplan ship with plan templates?
2. **Report Generation**: Should riotplan generate execution reports?
3. **Import/Export**: Should plans be exportable to other formats?
4. **Collaboration**: How do multiple people work on the same plan?

Part 14: Critical Review (Self-Assessment)

This section was added after reviewing the analysis for completeness. Several important considerations were missing.

Success Metrics

How do we know RiotPlan is working? Quantifiable targets:

Metric	Current State	Target with RiotPlan
Context recovery time	2+ hours/day	$<$ 10 minutes/day

Metric	Current State	Target with RiotPlan
Lost decision lookups	Can't find ~60%	Find 95%+
Documentation overhead	1-2 hours/day manual	< 15 min (automated)
Cross-project coordination	Mental gymnastics	Single dashboard view
Feedback preservation	0% (conversations lost)	100% (captured in records)
Plan discovery	Manual directory scanning	`riotplan dashboard` instant

Risk Assessment

Risk	Likelihood	Impact	Mitigation
Scope creep	High	High	R1-R29 is already large. Phase strictly. MVP = R1-R7.
Adoption friction	Medium	High	Must work with existing plans without migration. Progressive enhancement.
Over-engineering	Medium	Medium	Start with files on disk. No database. No server (initially).
Documentation debt	High	Medium	Documentation is Phase 0.5, not afterthought.
RiotPrompt integration complexity	Medium	Medium	Start with shared types only. Full integration later.
Attention tracking privacy	Low	High	All data local. No telemetry. User owns their data.

Explicitly Out of Scope (v1.0)

To prevent scope creep, these are **NOT** in initial release:

1. **Branching history** - Linear timeline only. No git-style branches.
2. **Real-time collaborative editing** - No live sync. Git provides collaboration (see below).
3. **Cloud sync** - All local. No server component in v1.
4. **AI plan generation** - RiotPlan manages plans, doesn't create them.
5. **Mobile app** - CLI and VS Code only. Dashboard is desktop web.
6. **Hosted sharing** - `riotplan share` creates files, not URLs (initially).
7. **Semantic search** - Keyword search initially. Semantic/NLP search is R29 (Phase 4-5).

R29: Semantic Search and RAG Support (FUTURE - Phase 4-5)

- ☐ Index plan content in local vector database
- ☐ `riotplan index` to build/update search index
- ☐ `riotplan search <query>` for semantic search
- ☐ `riotplan search --similar <file>` for similarity matching
- ☐ Support multiple embedding backends (local, OpenAI, Ollama)
- ☐ RAG integration: surface relevant content during plan creation
- ☐ Pattern discovery across plans

Rationale: As plans accumulate, finding similar patterns, phases, and contexts becomes difficult with keyword search. NLP/semantic search enables: - "Find plans that dealt with security concerns" - "Show feedback similar to this discussion" - "What did we learn about API design?"

Why not Phase 1: Requires embedding pipeline, vector DB, additional dependencies. Keyword search works for small collections. Implement when users have 20+ plans.

Collaboration via Git (Clarification)

"Single-user" means no built-in real-time sync—not no collaboration.

Plans stored in Git enable full team collaboration:

Need	Solution
Multiple contributors	Push/pull to shared repo
Review process	Pull requests
Feedback from others	Add files to `feedback/` directory
Conflict resolution	Git merge/rebase
Attribution	Git commit author

Team workflow example:

```
# Alice creates plan
riotplan plan init feature-x --template feature
git add . && git commit -m "plan: feature-x"
git push

# Bob adds feedback
git pull
riotplan feedback create "scope concerns"
git push

# Carol reviews via PR, team iterates
```

Multiple people CAN generate feedback—each contributor adds files to `feedback/`:

```
feedback/
  001-initial-scope-tim.md
  002-security-concerns-alice.md
  003-api-review-bob.md
```

Why Git, not built-in multi-user: - Teams already use Git for code; plans live alongside code - No new infrastructure to deploy - PR-based review is well-understood - Asynchronous deliberation is better than real-time editing for plans - Works offline, no vendor lock-in

Real-time collaboration may come in Phase 3+, but Git is sufficient for v1.

These may come in v2.0+ based on user feedback.

Privacy Considerations

RiotPlan captures sensitive data:

Data Type	Sensitivity	Consideration
Feedback records	High	Contains conversations, decisions, reasoning
Attention timeline	High	Records what user looked at and when
Plan content	Medium	May contain proprietary business logic
Evidence files	Medium	May contain confidential research

Principles: 1. **All data is local** - Nothing sent to servers 2. **User owns data** - Standard filesystem, not proprietary database 3. **Exportable** - Can always get data out in readable formats 4. **Optional tracking** - Attention tracking can be disabled 5. **Anonymization support** - R8 includes anonymization for sharing

Migration Path

For users with existing manual plan structures:

Phase 1: Recognition - RiotPlan recognizes existing `prompts/` directories - Works with plans that don't have all standard files - Adds missing files incrementally

Phase 2: Progressive Enhancement - `riotplan validate` shows what's missing - `riotplan upgrade` adds standard files without overwriting - Existing STATUS.md formats parsed best-effort

Phase 3: Full Integration - Once validated, full feature set available - No "big bang" migration required

What This Analysis Doesn't Cover

1. **Detailed technical specifications** - This is requirements, not implementation
2. **API documentation** - Will be created alongside code
3. **Test strategies** - Will be defined in implementation plan
4. **Performance requirements** - Not yet benchmarked
5. **Accessibility** - Dashboard should be accessible, but not detailed here

---

Conclusion

RiotPlan addresses a real problem: managing complex, multi-step, cross-session work is cognitively demanding. The manual `what-happened-in-*.md` files that prompted this analysis are themselves evidence that users need better tooling.

The observed patterns are consistent: - Plans have standard structure - Plans evolve through multiple revisions - Plans can spawn other plans - Plans can depend on external work - Users lose track when managing multiple plans

RiotPlan's goal is to formalize these patterns into tooling that: 1. Reduces cognitive overhead 2. Enables context recovery 3. Tracks dependencies 4. Supports the natural evolution of plans

The current stub implementations in `/Users/tobrien/gitw/kjerneverk/riotplan` and `/Users/tobrien/gitw/kjerneverk/riotplan-cli` provide a solid foundation of types and CLI structure. The next step is implementing the core functionality outlined in Phase 1.

Appendix: Feedback On This Analysis

This analysis document itself has undergone feedback and revision. The deliberation is preserved in:

- `feedback/001-deliberation-capture.md` - The conversation that added the "Philosophical Foundation" section
- `feedback/002-multimodal-and-historical.md` - Recognition of multi-modal potential and historical significance
- `feedback/003-human-experience-stories.md` - Request for human stories grounding the technical in lived experience
- `feedback/004-expanded-lifecycle-and-metaplans.md` - Major expansion: lifecycle phases, archive utility, metaplans, scope beyond code
- `feedback/005-inception-evidence-directory.md` - Adding `evidence/` directory for inception phase materials
- `feedback/006-revision-tracking-and-artifacts.md` - Embedded versioning, artifact packaging, RiotPrompt integration, prompt testing
- `feedback/007-language-and-metacognition.md` - Language corrections ("human suffering"), metacognition emphasis
- `feedback/008-plans-as-evolving-records.md` - Plans as evolving records of reasoning, not static documents
- `feedback/009-cross-tool-and-attention-tracking.md` - Cross-tool context switching, attention tracking at 100x speed
- `feedback/010-contexts-and-rendering-modes.md` - Plan contexts (work/personal), rendering modes (full/current)
- `feedback/011-ecosystem-and-global-guidance.md` - Cross-project awareness, global guidance and style configuration
- `feedback/012-project-structure-and-packages.md` - Multiple packages, VS Code extension, React dashboard
- `feedback/013-repository-setup-proposal.md` - Monorepo vs multi-repo, directory structure proposal

- `feedback/014-documentation-requirements.md` - Philosophy doc, human stories doc, GitHub Pages, "The Book"
- `feedback/015-transition-to-plan-creation.md` - Marking the transition from analysis to plan creation
- `feedback/016-critical-review-and-gaps.md` - Self-assessment: success metrics, risks, out-of-scope, privacy, migration
- `feedback/017-devils-advocate-review-pattern.md` - External critical review pattern: multi-AI workflow for devil's advocate analysis
- `feedback/018-history-pruning-and-cleanup.md` - Response to critique: history lifecycle management, cleanup at milestones
- `feedback/019-api-first-and-open-adoption.md` - API-first architecture, background daemon, impact over revenue
- `feedback/020-audience-aware-documentation.md` - Two paths: practical (no jargon) and philosophical (full depth)
- `feedback/021-overhead-vs-current-chaos.md` - Overhead is relative; the tool automates existing manual work
- `feedback/022-research-references-and-terminology.md` - Academic grounding; avoid "metacognitive" in user-facing docs
- `feedback/023-tooling-support-at-each-stage.md` - Lifecycle-aware commands; `riotplan review` initiates feedback capture
- `feedback/024-extensible-execution-providers.md` - Pluggable providers (Anthropic, OpenAI, Human, CI/CD); not tied to one tool
- `feedback/025-completion-retrospectives.md` - Plans should teach; RETROSPECTIVE.md, OUTCOME.md, AI self-reflection
- `feedback/026-starter-templates.md` - Ship templates (feature, bugfix, content, etc.); wizard mode; don't start from scratch
- `feedback/027-collaboration-via-git.md` - "Single-user" means no real-time sync; Git IS the collaboration layer
- `feedback/028-explainability-and-communication.md` - **CRITICAL**: Explain simply; audience-specific messaging; scenarios > jargon
- `feedback/029-templates-blank-slate-problem.md` - Templates are Phase 1 priority; blank slates kill adoption
- `feedback/030-semantic-search-and-rag.md` - Future: NLP search, vector DB, RAG for pattern discovery (Phase 4-5)
- `feedback/031-research-references-document.md` - Created evidence/research-references.md with academic grounding

Revision History

Version	Date	Changes	Feedback Source
v0.1	2026-01-14	Initial analysis from metadata files	N/A
v0.2	2026-01-14	Added philosophical foundation, feedback capture requirements	`feedback/001-deliberation-capture.md`

Version	Date	Changes	Feedback Source
v0.3	2026-01-14	Added multi-modal sources, historical significance	`feedback/002-multimodal-and-historical.md`
v0.4	2026-01-14	Added human experience stories (before/after)	`feedback/003-human-experience-stories.md`
v0.5	2026-01-14	Expanded lifecycle (7 phases), metaplans, archive, forking, scope beyond code	`feedback/004-expanded-lifecycle-and-metaplan`
v0.6	2026-01-14	Added `evidence/` directory for inception phase materials	`feedback/005-inception-evidence-directory.md`
v0.7	2026-01-14	Embedded versioning, artifact packaging, RiotPrompt cross-pollination, prompt testing	`feedback/006-revision-tracking-and-artifacts`
v0.8	2026-01-14	Language corrections, metacognition emphasis, plans as evolving records	`feedback/007, 008`
v0.9	2026-01-14	Cross-tool capture, attention tracking, contexts, rendering modes	`feedback/009, 010`
v0.10	2026-01-14	Ecosystem awareness, global guidance, project structure, multiple packages	`feedback/011, 012`
v0.11	2026-01-14	Repository setup proposal, documentation requirements, GitHub Pages	`feedback/013, 014`
v0.12	2026-01-14	Phase 0 transition point	`feedback/015`
v0.13	2026-01-14	**Critical review**: success metrics, risks, scope, privacy, migration	`feedback/016`

Version	Date	Changes	Feedback Source
v0.14	2026-01-14	**External critical review pattern**: multi-AI devil's advocate workflow, R24	`feedback/017`
v0.15	2026-01-14	**History lifecycle management**: cleanup at milestones, R25	`feedback/018`
v0.16	2026-01-14	**API-first architecture**: background daemon, open spec, impact over revenue	`feedback/019`
v0.17	2026-01-14	**Audience-aware documentation**: practical path vs philosophical path	`feedback/020`
v0.18	2026-01-14	**Overhead trade-off**: tool overhead < chaos overhead	`feedback/021`
v0.19	2026-01-14	**Research references + terminology**: academic grounding; avoid "metacognitive" jargon	`feedback/022`
v0.20	2026-01-14	**Lifecycle-aware commands**: `riotplan review` initiates feedback; `execute` auto-updates status	`feedback/023`
v0.21	2026-01-14	**Extensible execution providers**: R26; pluggable backends; humans are a valid provider	`feedback/024`

Version	Date	Changes	Feedback Source
v0.22	2026-01-14	**Completion retrospectives**: R27; RETROSPECTIVE.md, OUTCOME.md, AI self-reflection	`feedback/025`
v0.23	2026-01-14	**Starter templates**: R28; built-in templates for feature, bugfix, content; wizard mode	`feedback/026`
v0.24	2026-01-14	**Collaboration clarification**: Git IS the collaboration layer; multiple contributors via feedback/	`feedback/027`
v0.25	2026-01-14	**Explainability principle**: CRITICAL; elevator pitches; 5-level doc hierarchy; scenarios > jargon	`feedback/028`
v0.26	2026-01-14	**Templates Phase 1**: Blank slates kill adoption; templates are required, not nice-to-have	`feedback/029`
v0.27	2026-01-14	**Semantic search (future)**: R29; NLP search, vector DB, RAG for pattern discovery (Phase 4-5)	`feedback/030`
v0.28	2026-01-14	**Research references**: Created evidence/research-references.md with academic grounding	`feedback/031`

This self-referential approach—using the very patterns we're proposing to document the proposal itself—demonstrates the viability of the feedback capture mechanism.

What We're Building In Real-Time

This analysis and its feedback records constitute: 1. **A design document** - What RiotPlan should be 2. **A historical record** - How the design evolved 3. **A proof of concept** - The feedback mechanism in action 4. **Plan history** - Being written as it happens 5. **Human story** - Why this matters to real people 6. **Metacognitive framework** - How we think about structured work

The Evolution Traced

Feedback	Core Addition	Conceptual Shift
001	Feedback capture	Plans include their deliberation history
002	Multi-modal input	Feedback comes from anywhere
003	Human stories	This is about cognitive liberation
004	Lifecycle + Metaplans	Plans are knowledge artifacts, not disposable instructions
005	Evidence directory	Inception is grounded in observed reality
006	Versioning + Artifacts	Plans are portable, self-contained, testable units
007-008	Metacognition + Evolving records	The conversation is the value; plans are threads of thought
009-010	Attention + Contexts	Track where your head is; support how life really works
011-012	Ecosystem + Project structure	Plans exist within ecosystems; design for modularity
013-014	Repo setup + Documentation	Build for others to use; documentation is a core deliverable

The feedback records trace the evolution from "tool for managing AI coding tasks" to "metacognitive framework for structured deliberation on complex work."

Records 001-006 established the foundation: feedback capture, evidence, versioning, artifacts.

Records 007-008 deepened the philosophy: the conversation is where value lives; plans are evolving records of reasoning.

Records 009-010 addressed practical reality: attention tracking at 100x speed; contexts for work vs personal life.

Records 011-012 expanded the architecture: cross-project awareness; six packages (core, cli, vscode, dashboard, server, renderer).

Records 013-014 prepared for launch: repository structure for user to create; documentation as Phase 0.5.

Record 017 introduced the devil's advocate pattern: sending plans to external AI models (like ChatGPT Pro with Deep Research) for critical review, grounding concepts in academic literature, and capturing the multi-AI workflow as valuable deliberation.

This morning session produced 23 feedback records (007-014, 017-031), demonstrating both the volume of valuable deliberation that occurs and the importance of capturing it. Without these records, this morning's ideas would have been lost.

Record 018 addressed the critique that history could become unwieldy—proposing milestone-based cleanup with Git as the safety net.

Record 019 clarified the architecture (API-first, background daemon) and the project's motivation: impact over revenue.

Record 020 proposed audience-aware documentation: a practical path (no jargon, just tools) alongside a philosophical path (the full picture).

Record 021 contextualized overhead concerns: the tool automates work already being done manually; its overhead is less than the chaos it replaces.

Record 022 extracted research references from the ChatGPT review and established terminology guidance: use "metacognitive" only in philosophy docs; everywhere else, use plain language like "reasoning trace" or "decision log."

Record 023 identified lifecycle-aware commands: `riotplan review` initiates feedback capture sessions; `riotplan execute` auto-updates STATUS.md. The CLI becomes a workflow guide, not just utilities.

Record 024 established extensible execution providers: plans shouldn't be tied to one tool. Different steps need different capabilities—Claude Code for implementation, OpenAI for research, humans for approvals, CI/CD for automation. "Humans are a valid provider."

Record 025 added completion retrospectives: plans shouldn't just end—they should teach. `RETROSPECTIVE.md` captures phase-by-phase reflection, key decisions, and recommendations. `OUTCOME.md` summarizes deliverables. AI self-reflection captures confidence and difficulty when AI executes steps.

Record 026 introduced starter templates: don't make users start from scratch. Ship templates for common scenarios (feature, bugfix, refactor, content, etc.) with a wizard mode for interactive guided creation. Templates encode best practices; metaplans are battle-tested refinements.

Record 027 clarified the collaboration model: "single-user" means no built-in real-time sync—not no collaboration. Git IS the collaboration layer. Plans stored in Git enable multiple contributors via PRs, and multiple people can add feedback files. Asynchronous deliberation via Git is better than real-time editing for plans.

Record 028 (CRITICAL) elevated explainability to a core design principle: "If we can't explain it simply, we don't understand it well enough." Added elevator pitches for different audiences, 5-level documentation hierarchy, real scenario walkthroughs, and terminology cleanup. Communication isn't a nice-to-have—it's existential for adoption.

Record 029 reinforced templates as Phase 1 priority: "Blank slates kill adoption." From a product standpoint, templates are required, not nice-to-have. Even a simple folder of example plans addresses the problem. Templates teach by example and get users to value faster.

Record 030 added R29: Semantic Search and RAG Support as a future capability (Phase 4-5). NLP search on plan content enables pattern discovery, similarity matching, and RAG for plan creation. Explicitly marked as "not Phase 1" but documented for when plan collections grow large.

Record 031 created `evidence/research-references.md` extracting all academic references from the ChatGPT critical review. Provides academic grounding for RiotPlan concepts—IBIS, ADRs, metacognitive tools, Plan-Then-Execute study—with citations for documentation.

Phase 0 Complete: Ready for Plan Creation

This analysis has reached its natural conclusion. After 31 feedback sessions across two days, we have:

Deliverables from Phase 0

Deliverable	Location	Purpose
This Analysis	`RIOTPLAN_ANALYSIS.md`	Comprehensive design document
Evidence Files	`what-happened-*.md` (10 files)	Case studies that informed design
External Review	`evidence/chatgpt-critical-review-2026-01-14.md`	Devil's advocate analysis from ChatGPT 5.2 Pro
Research References	`evidence/research-references.md`	Academic grounding for RiotPlan concepts
Feedback Records	`feedback/*.md` (31 files)	Deliberation history
PDF Export	`RIOTPLAN_ANALYSIS.pdf`	Shareable artifact

Next Actions

1. **User**: Review proposed repository structure (Part 7)
2. **User**: Create GitHub repository with monorepo layout
3. **Then**: Create implementation plan using this analysis as `evidence/`
4. **Then**: Execute Phase 1 (core + cli packages)

The Recursive Proof

We designed a system for: - Capturing evidence during inception (10 `what-happened-*.md` files) - Preserving deliberation through feedback (15 feedback records) - Transitioning from inception to plan creation (this moment)

And we used that system to design itself. The plan for building RiotPlan will reference this analysis as its evidence base—demonstrating that the approach works.

Document created: 2026-01-14 Last updated: 2026-01-14 (v0.28) Based on analysis of 10 evidence files and multiple plan directories Evidence: /Users/tobrien/gitw/kjerneverk/prompts/reverse-engineer-riotplan/what-happened-.md External review: evidence/chatgpt-critical-review-2026-01-14.md (ChatGPT 5.2 Pro Deep Research) Research references: evidence/research-references.md (academic grounding for RiotPlan concepts) Feedback captured in: feedback/ (31 records documenting the evolution)* **Status: Phase 0 Complete - Ready for Plan Creation***